《算法分析与设计》

西南交通大学
Southwest Jiaotong University

动态规划
Dynamic Programming

计算机与人工智能学院
李晓航
xhli_scce@swjtu.edu.cn

Last: 2026-05

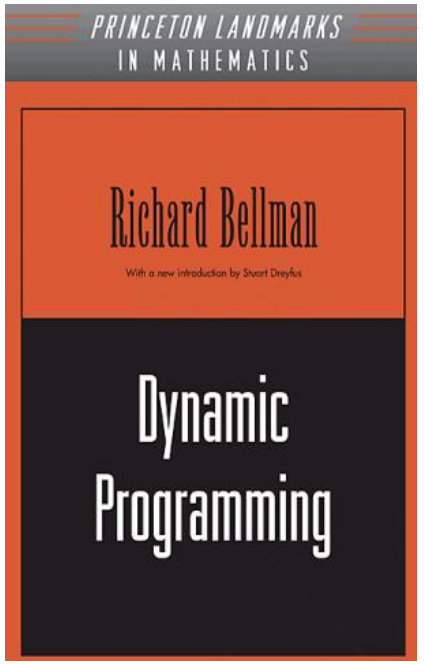
一、动态规划概述

- 1.1 动态规划的起源
- 1.2 动态规划的几个概念
- 1.3 多段图的最短路径问题
- 1.4 动态规划算法特点
- 1.5 动态规划问题的求解步骤

1.1 动态规划的起源

20世纪50年代初美国数学家R.E.Bellman等人在研究多阶段决策过程(multistep decision process)的优化问题时，提出了著名的最优化原理(principle of optimality)，把多阶段过程转化为一系列单阶段问题，利用各阶段之间的关系，逐个求解，创立了解决这类过程优化问题的新方法——动态规划。





3

动态规划的起源（续）

动态规划（dynamic programming）是运筹学的一个分支，是求解决策过程（decision process）最优化的数学方法。

动态规划问世以来，在经济军事、生产调度、工程技术和最优控制等方面得到了广泛的应用。例如最短路线、库存管理、资源分配、导弹拦截、货物装载等问题。



1.2 动态规划的几个概念

1）可行解、目标函数和最优化问题

- ✓ 可行解：假设某问题的解由一个集合的子集组成，这个子集必须满足某些事先给定的条件（即约束条件），满足约束条件的解称为问题的可行解。
- ✓ 目标函数：满足约束条件的可行解可能不只一个，为了衡量这些可行解的优劣，事先给出一定的标准，这些标准通常以函数的形式给出，这些函数称为目标函数（也叫优化函数）。
- ✓ 最优化问题：使目标函数取得极值（极大或极小）的可行解称为最优解，这类问题就称为**最优化问题**。

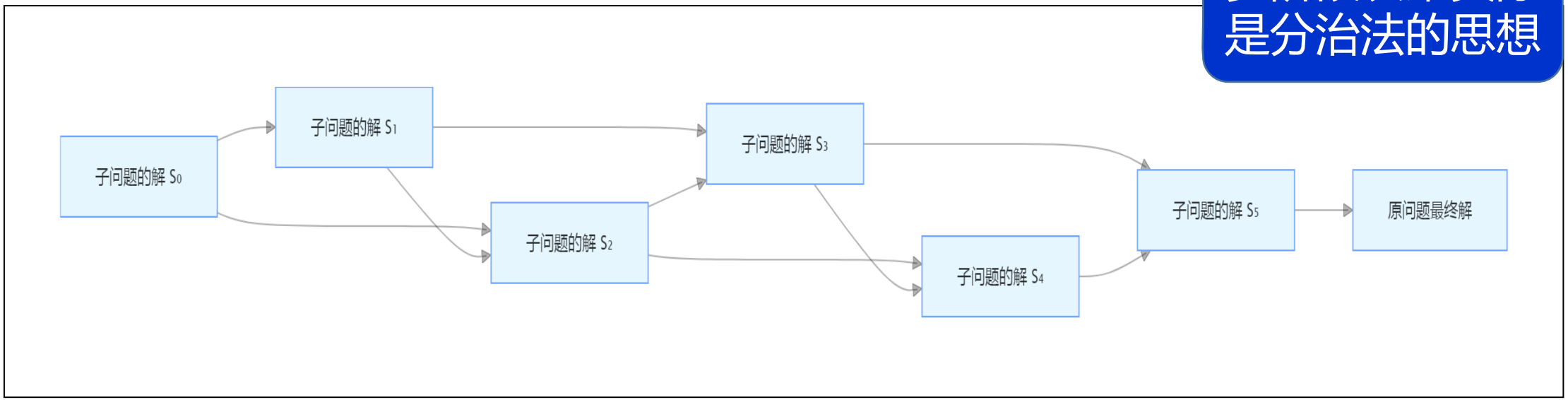
动态规划通常用于解决最优化问题

动态规划的几个概念（续）

2）多阶段决策过程

对于一个具有 n 个输入的最优化问题，如果其求解过程可以划分为若干个阶段（子问题），每一阶段的决策仅依赖于前一阶段的状态（即前面某些子问题的解），由决策所采取的动作使状态发生转移，成为下一阶段决策的依据。从而，一个决策序列在不断变化的状态中产生。这个决策序列产生的过程称为**多阶段决策过程**。

多阶段决策实际是分治法的思想



动态规划的几个概念（续）

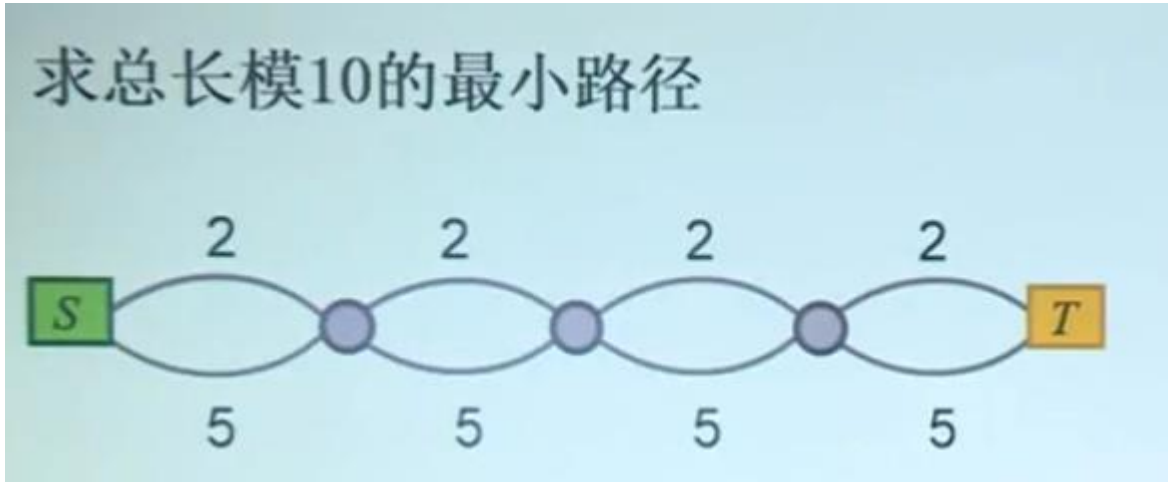
3）最优化原理（Optimal Principle）

- ✓ 最优化原理指无论决策过程的初始状态和初始决策是什么，其余的决策都必须相对于初始决策所产生的当前状态，构成一个最优决策序列。
- ✓ 即在多阶段决策中，各子问题的解只与它前面的子问题的解有关，并且各子问题的解都是相对于当前子问题的最优解，原问题的最优解是由各个子问题的最优解构成。
- ✓ 如果一个问题满足最优化原理通常称此问题具有最优子结构性质。
- ✓ 不满足最优化原理的问题，不能用动态规划算法求解。

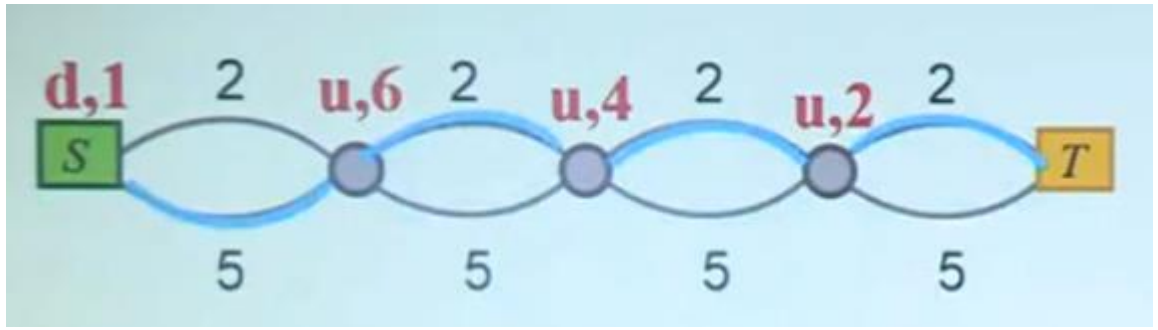
动态规划的几个概念（续）

一个不满足最优化原理问题

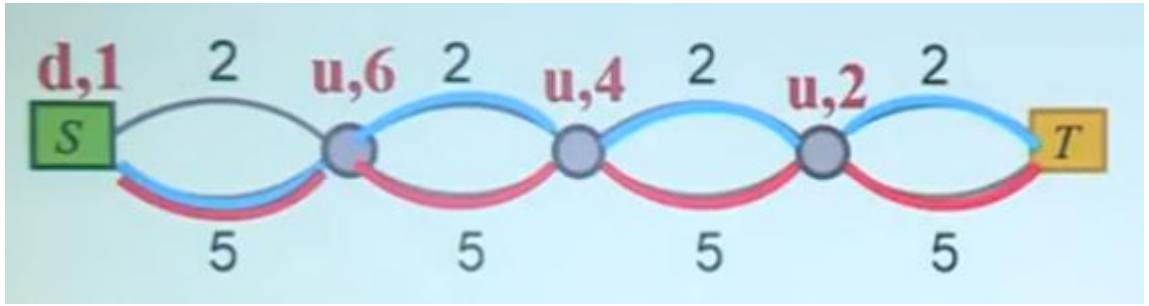
余数最短路径问题



按最优化原理得出的最短路径（d=1）



实际的最短路径（d=20 mod 10=0）



该问题不满足最优化原理，不能用动态规划。

动态规划的几个概念（续）

4）无后效性（No Aftereffect）

指如果某一阶段（子问题）的状态（解）一旦确定，后续阶段的决策不会再改变该状态的值，也不影响之前阶段的状态。

即过去的状态只由过去的决策决定，和未来的决策无关；一旦走到某个状态，就可以把它当成新的起点。

不具备无后效性的问题，不能用动态规划算法求解。

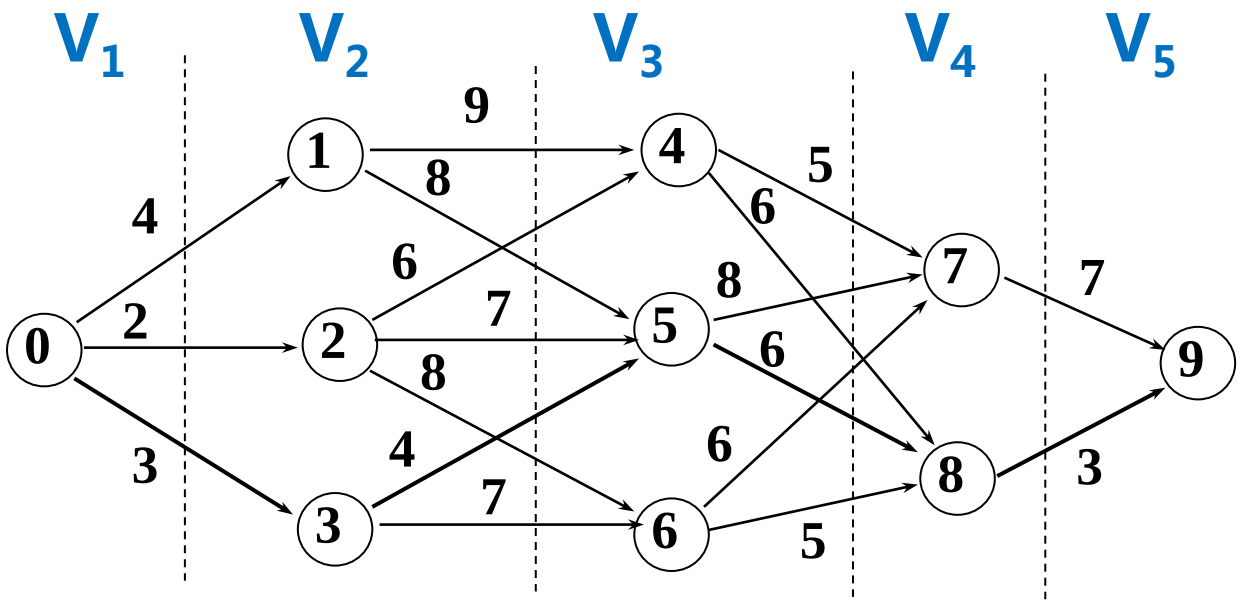
动态规划的几个概念（续）

附：动态规划算法为何效率高？

- ✓ 回溯法是在解空间树中遍历所有的决策路径，而解空间的规模通常是指数级别（ $O(m^n)$ ），因此效率低下。
- ✓ 动态规划法是把原问题按变化维度拆成独立的子问题，依靠最优子结构 + 无后效性+状态转移方程+dp数组，用已算出的子问题的解，逐步推出更大问题的解（直到原问题的解），不会重复计算子问题。
- ✓ 因此动态规划算法的时间复杂度基本等价于子问题数量，而子问题数量就是原问题各变化维度（有限离散的）数量相乘，是多项式级别。

1.3 多段图的最短路径问题

设图G=(V, E)是一个带权有向连通图，如果把顶点集合 V 划分成 k 个互不相交的子集V_i（$2\le k\le n$，$1\le i\le k$），使得 E 中的任何一条边（u，v），必有 $u\in V_i$，$v\in V_{i+m}$（$1\le i<k$，$1<i+m\le k$），则称图 G 为多段图，其中 $s\in V_1$ 为源点，$t\in V_k$ 为终点。多段图的最短路径问题是求从源点到终点的最小代价路径。

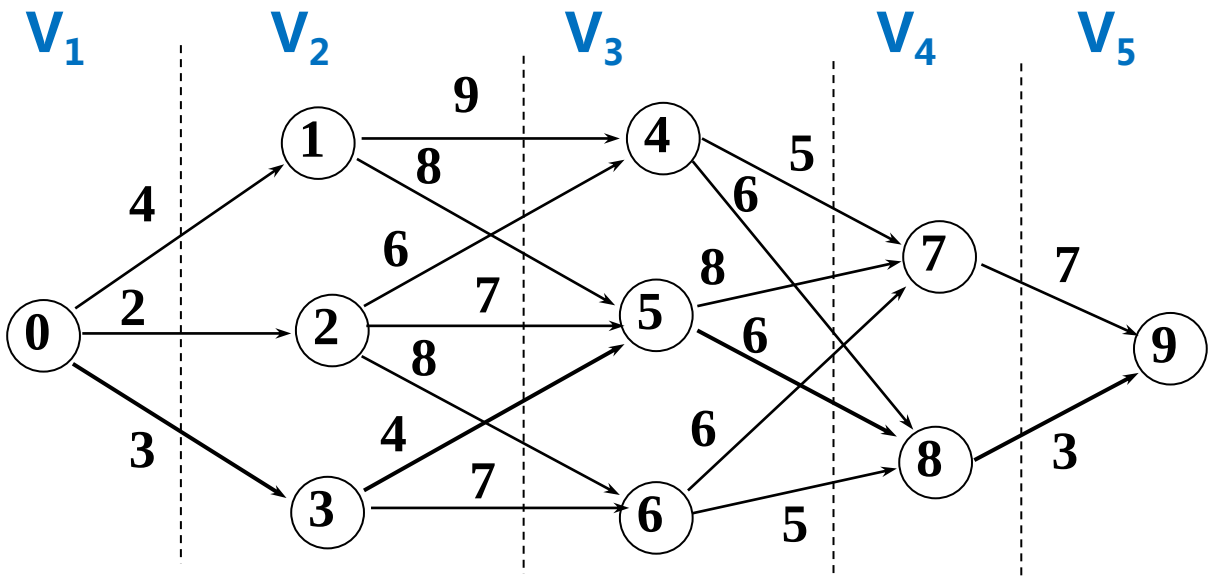


一个多段图

多段图的最短路径问题（续）

由于多段图将顶点划分为 k 个互不相交的子集，所以，多段图划分为 k 段，每一段包含顶点的一个子集。

不失一般性，将多段图的顶点按照段的顺序进行编号，同一段内顶点的相互顺序无关紧要。假设图中的顶点个数为 n，则源点 s 的编号为 0，终点 t 的编号为 n-1，并且，对图中的任何一条边(u, v)，顶点 u 的编号小于顶点 v 的编号。



一个多段图

多段图的最短路径问题（续）

对多段图的边(u, v)，用 c_{uv} 表示边上的权值，将从源点 s 到终点 t 的最短路径记为 d(s, t)，则从源点 0 到终点 9 的最短路径 d(0, 9) 由下式确定：

$$d(0, 9)=\min\{ c_{01}+d(1, 9),\ c_{02}+d(2, 9),\ c_{03}+d(3, 9) \}$$

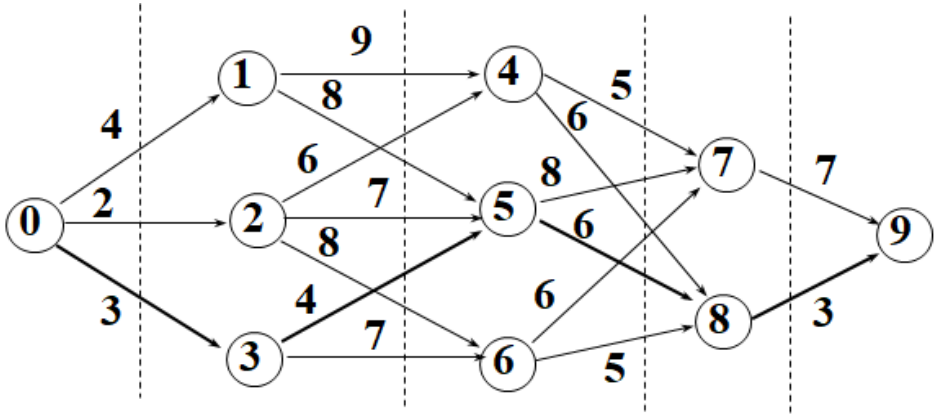
这是最后一个阶段的决策，它依赖于d(1, 9)、d(2, 9)和d(3, 9)的计算结果，而

$$d(1, 9)=\min\{c_{14}+d(4, 9), c_{15}+d(5, 9)\}$$

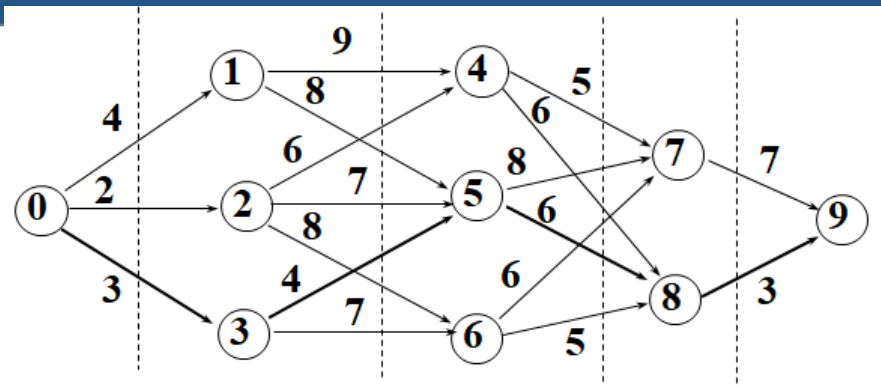
$$d(2, 9)=\min\{c_{24}+d(4, 9), c_{25}+d(5, 9), c_{26}+d(6, 9)\}$$

$$d(3, 9)=\min\{c_{35}+d(5, 9), c_{36}+d(6, 9)\}$$

这一阶段的决策又依赖于d(4, 9)、d(5, 9)和d(6, 9)的计算结果：



多段图的最短路径问题（续）



$d(4, 9)=\min\{c_{47}+d(7, 9), c_{48}+d(8, 9)\}$

$d(5, 9)=\min\{c_{57}+d(7, 9), c_{58}+d(8, 9)\}$

$d(6, 9)=\min\{c_{67}+d(7, 9), c_{68}+d(8, 9)\}$

这一阶段的决策依赖于 d(7, 9) 和 d(8, 9) 的计算，而d(7, 9)和d(8, 9)可以直接获得（括号中给出了决策产生的状态转移）：

$d(7, 9)=c_{79}=7$ (7→9)

$d(8, 9)=c_{89}=3$ (8→9)　（至此完成了“自顶向下”的分析）

然后再自底向上迭代，通过子问题得到原问题的解，有：

$d(6, 9)=\min\{c_{67}+d(7, 9), c_{68}+d(8, 9)\}=\min\{6+7, 5+3\}=8$ (6→8)

$d(5, 9)=\min\{c_{57}+d(7, 9), c_{58}+d(8, 9)\}=\min\{8+7, 6+3\}=9$ (5→8)

$d(4, 9)=\min\{c_{47}+d(7, 9), c_{48}+d(8, 9)\}=\min\{5+7, 6+3\}=9$ (4→8)

多段图的最短路径问题（续）

$d(3, 9)=\min\{c_{35}+d(5, 9), c_{36}+d(6, 9)\}=\min\{4+9, 7+8\}=13$ (3→5)

$d(2, 9)=\min\{c_{24}+d(4, 9), c_{25}+d(5, 9), c_{26}+d(6, 9)\}=\min\{6+9, 7+9, 8+8\}=15$ (2→4)

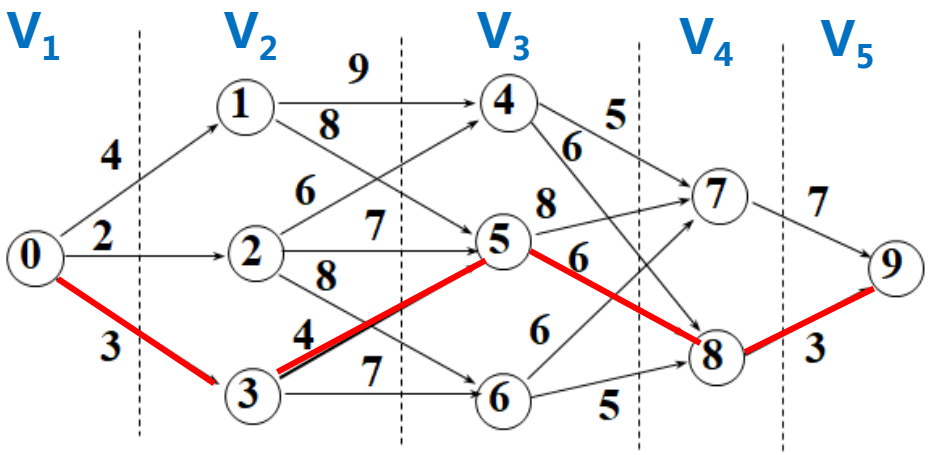
$d(1, 9)=\min\{c_{14}+d(4, 9), c_{15}+d(5, 9)\}=\min\{9+9, 8+9\}=17$ (1→5)

$d(0, 9)=\min\{c_{01}+d(1, 9), c_{02}+d(2, 9), c_{03}+d(3, 9)\}=\min\{4+17, 2+15, 3+13\}=16$ (0→3) （注：d(0,9)就是原始问题）

最后，得到最短路径为0→3→5→8→9，长度为16。

注：上述状态数组 d 可以用一维数组

思考：对一般的图能否用上述方法求解最短路径？



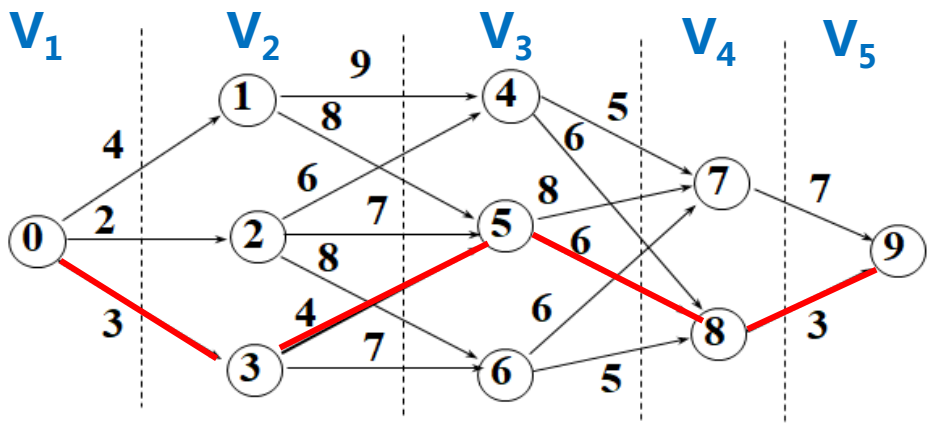
多段图的最短路径问题（续）

从以上过程可以看出，采用动态规划算法解决多阶段决策问题，其本质是分治算法。

即将含四个阶段 V_1- V_2-V_3-V_4-V_5 的原问题决策，先分解为简单的单阶段 V_1-V_2 和三阶段 V_2-V_3-V_4-V_5 的决策；而 V_2-V_3-V_4-V_5 的决策又分解为单阶段 V_2-V_3 和两阶段 V_3-V_4-V_5 的决策；V_3-V_4-V_5 的决策又分解为两个单阶段 V_3-V_4 和 V_4-V_5 的决策，最后求解 V_4-V_5 的决策。

实现时可进行逆向求解，即逆序求解V_4-V_5、 V_3-V_4-V_5、 V_2-V_3-V_4-V_5 ，最后求解 V_1- V_2-V_3-V_4-V_5 ，便得到了全过程的最短路径及最短距离，同时附带得到了一组最优结果（即各阶段的点到终点 9 的最优结果）。

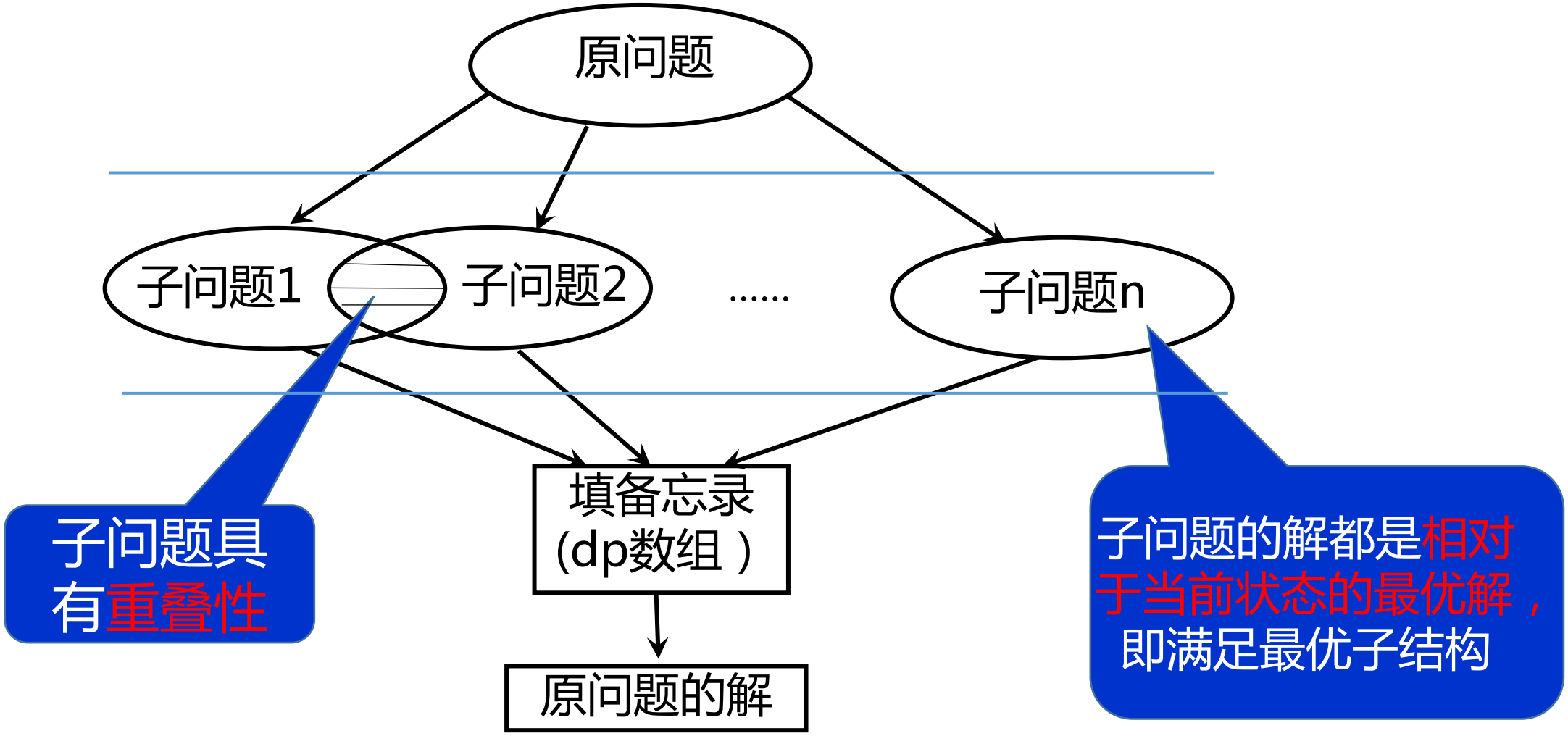
该算法将单阶段求得的最短距离进行存储，以备后续调用（备忘录）。同时，算法只需要一个双重循环就可以实现各个阶段各个点的最短路径求解，因此算法的时间复杂度为$O(n^2)$。



1.4 动态规划算法特点

- 动态规划的本质仍然是“分治”算法
  - 即动态规划也是将待求解问题分解成若干个子问题（分解），先求解子问题，再从子问题的解得到原问题的解（合并）。
- 动态规划分解得到的子问题往往相互重叠（重叠子问题）
  - 如果用标准分治法来解这类问题，则重叠子问题会被重复计算多次，以至于最后解决原问题需要耗费指数类型的时间复杂度。
  - 动态规划算法的思想之一就是用“备忘录”保存已解决的子问题的解，在需要时直接获取已保存的子问题解，以避免重复计算，从而得到多项式类型的时间复杂度的算法。
- 动态规划只能解决满足“最优化原理”和“无后效性”的问题。

动态规划算法的适用问题



动态规划算法特点（续）

- 动态规划算法解决的问题类型：
  - 1、最优化问题：即计算目标函数最优解的问题。
  - 2、计数问题：即计算求解某个问题的方法总数的问题。
  - 3、可达性问题：即判断某个问题是否存在解的问题。
- 可以把动态规划看做是分治法的特例，只专注解决最优化、计数、可达性这三类带重叠子问题的题目。

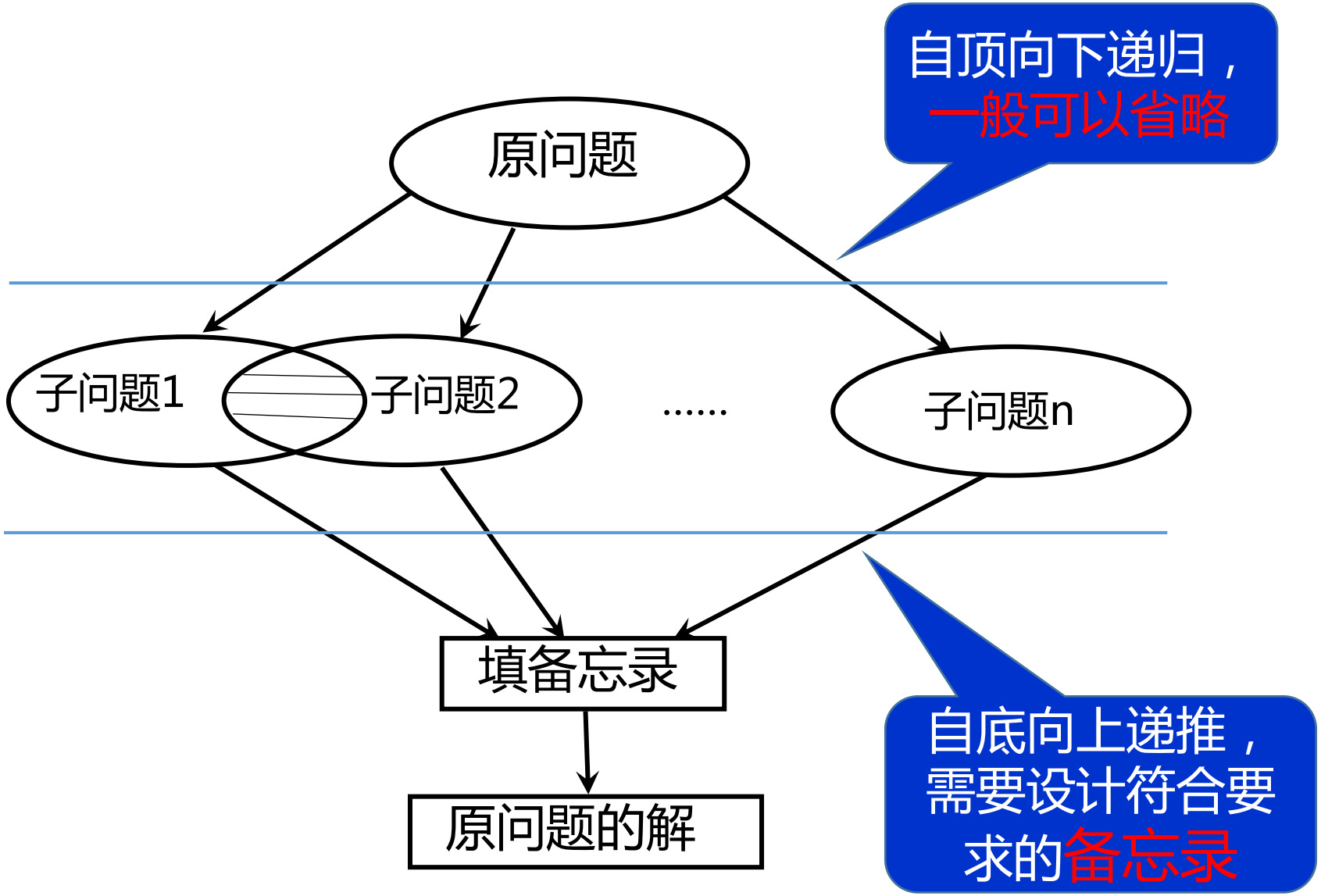
1.5 动态规划问题的求解步骤

1）分析最优解的结构，定义子问题
2）给出递推方程（状态转移方程）
3）自底向上（或自顶向下）计算局部最优解的值
4）根据局部最优解的值构造全局最优解
5）最优方案的追踪

注：递推是“自底向上”计算，带备忘递归是“自顶向下”计算。动态规划算法通常直接采用递推法（效率更高）。



动态规划问题的求解步骤（续）



求解动态规划问题的五步曲（《代码随想录》）

- 1、确定dp数组的值以及dp数组下标的含义
  - dp数组的值即子问题的解， dp数组下标即子问题的划分。
  - 根据原问题dp数组有有一维、二维甚至更多维的情况。
- 2、确定递推公式
  - 递推公式即dp数组中值的变化（状态转移）公式。
- 3、对dp数组进行初始化
  - 即确定递推的起点值。
- 4、确定遍历顺序
  - 在遍历时要确保先计算规模较小的子问题的解，这样在解决后续规模较大的子问题时，才能利用比它更小的所有子问题的解。
- 5、举例推导dp数组

二、备忘录方法

- 2.1 备忘录方法概述
- 2.2 斐波纳契数列
- 2.3 二项式系数

2.1 备忘录方法概述

- 备忘录方法是动态规划算法的简单变形。
- 如果原何题仅由交叠的子问题组成，而不进行最优决策的求解，则动态规划算法的核心就简化为分治和解决冗余。
- 备忘录方法同分治法类似，先采用通用的递推式将原问题进行分解，而后分别求解这些子问题，并为每个处理过的子问题建立备忘录，即建立一张表记录子向题的求解结果，以备需要时查看，从而避免了重复求解。
- 由于备忘录方法不涉及最优决策，因此既可采用“自顶向下”的递归方式，也可采用“自底向上”的递推方式（常用的动态规划算法主要采用自底向上的递推）。

2.2 带备忘录的斐波纳契数列算法

递推函数

$$F(n) = \begin{cases} 1 & \text{if } n = 0 \text{ or } 1 \\ F(n\text{-}1) + F(n\text{-}2) & \text{if } n > 1 \end{cases}$$

通过递推方程指导建立备忘录

自底向上的递推（更高效）

```
1    F[0] = F[1] ← 1
2    for i ← 2 to n do
3        F[i] ← F[i-1] + F[i-2]
4    return F[n]
```

直接利用备忘录数组 F
自底向上解决问题

自顶向下的递归

```
int TopDownFibonacci (int n, int F[0..n])
{
  for i = 0 to n do F[i] = -1;
  F[0] = 0;
  F[1] = 1;
  return Fibonacci(n,F);
}
int Fibonacci (int n, int F[0..n])
{
  if (F[n] <> -1) return F[n];
  int u = Fibonacci (n-1,F) + Fibonacci (n-2,F);
  F[n] = u;
  return F[n];
}
```

备忘录数组 F

初始化 F 为“未计算”

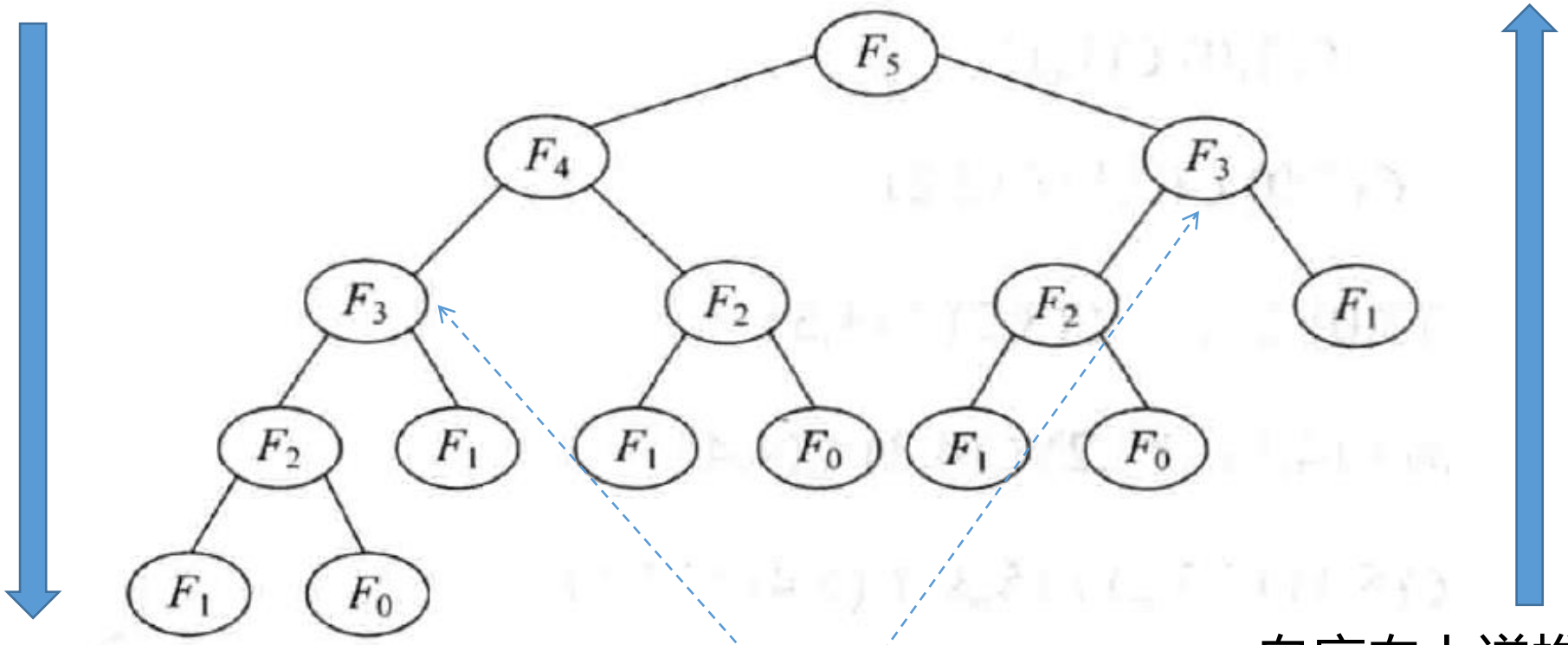
备忘录有合法值，直接取出，不再重复计算

由递归过程分解问题

将子问题结果存入备忘录，表示已计算

带备忘录的斐波纳契数列算法（续）

计算 F(5) 的递归树



自顶向下递归，可以省略

重复子问题

自底向上递推。要注意解决子问题的顺序，保证小的子问题先被解决

2.3 带备忘录的二项式系数计算

二项式公式

$$(a+b)^n = C(n,0)a^n + \cdots + C(n,k)a^{n-k}b^k + \cdots + C(n,n)b^n$$

二项式系数例子（也叫杨辉三角）

$(a+b)^1$ $C(1,0)$ $C(1,1)$

$(a+b)^2$ $C(2,0)$ $C(2,1)$ $C(2,2)$

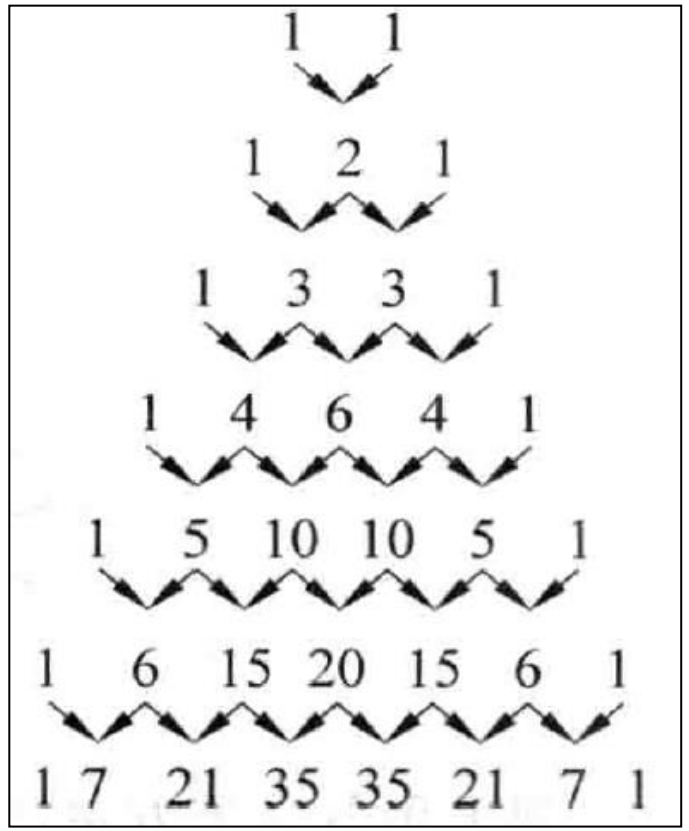
$(a+b)^3$ $C(3,0)$ $C(3,1)$ $C(3,2)$ $C(3,3)$

$(a+b)^4$ $C(4,0)$ $C(4,1)$ $C(4,2)$ $C(4,3)$ $C(4,4)$

$(a+b)^5$ $C(5,0)$ $C(5,1)$ $C(5,2)$ $C(5,3)$ $C(5,4)$ $C(5,5)$

$(a+b)^6$ $C(6,0)$ $C(6,1)$ $C(6,2)$ $C(6,3)$ $C(6,4)$ $C(6,5)$ $C(6,6)$

$(a+b)^7$ $C(7,0)$ $C(7,1)$ $C(7,2)$ $C(7,3)$ $C(7,4)$ $C(7,5)$ $C(7,6)$ $C(7,7)$



带备忘录的二项式系数计算（续）

二项式系数的递推函数和初始值

$$C(n,k)=\begin{cases}1 & k=0 \text{ 或 } k=n \\ C(n-1,k-1)+C(n-1,k) & n>k>0\end{cases}$$

因此计算 C(n,k) 的问题可以表示为计算 C(n-1,k-1) 和 C(n-1,k) 这两个较小的重叠子问题。可以采用备忘录方法求解。

根据递推公式，可以把二项式系数保存在 n 行、n+1列的二维数组中，并且只需存放数组的下三角部分即可（通过递推函数指导建立备忘录）。

n \ k	0	1	2	…	n
1	1	1			
2	1	C(1,0)+C(1,1)	1		
i	1	C(i-1, 0)+C(i-1,1)	C(i-1, 1)+C(i-1, 2)	1	
n	1	C(n-1, 0)+C(n-1, 1)	C(n-1, 1)+C(n-1, 2)	…	1

带备忘录的二项式系数计算（续）

自顶向下的递归

```
int TopDownBinomial (int n, int k, int C[0..n][ 0..n])
{
    for i = 0 to n do
        for j = 0 to min(i,k) do  //只存储下三角矩阵
            if (j = = 0 || i = = j) C[i][j] = 1;
            else C[i][j] = 0;
    return Binomial (n,k,C)
}
int Binomial(int n,int k, int C[0..n][ 0..n])
{
    if (C[n][k]> 0) return C[n][k];
    int result = Binomial(n-1,k,C) + Binomial(n-1,k-1,C);
    C[n][k] = result;
    return C[n][k];
}
```

备忘录数组 C

计算备忘录C的初值

初始化备忘录C

备忘录有合法值，直接取出

将当前的子问题结果存入备忘录

带备忘录的二项式系数计算（续）

自底向上的递推方式

```
int BottomUpBinomial(int n, int k, int C[0..n][0..n])
{
    for i = 0 to n do
        for j = 0 to min(i,k) do //
            if (j == 0 || i == j) C[i][j] = 1;
            else                  C[i][j] = C[i-1][j-1] + C[i-1][j];
    return C[n][k];
}
```

备忘录二维数组 C

直接递推计算。
注意递推顺序，保证先解决小问题

其内层循环执行的总次数为

$$1+2+3+4+\cdots+k+\underbrace{(k+1)+(k+1)+\cdots(k+1)}_{n-k+1\text{个}}$$

比递归法更高效的算法

化简后结果

$$\frac{k(k+1)}{2}+(n-k+1)(k+1)=\frac{(2n-k+2)(k+1)}{2}\in\Theta(nk)$$



三、常见动态规划问题

- 3.0 零钱兑换问题
- 3.1 最大子段和
- 3.2 数塔问题
- 3.3 矩阵连乘
- 3.4 最长公共子序列
- 3.5 0-1背包问题
- 3.6 最优二叉查找树

3.0 零钱兑换问题

- 零钱兑换问题包含两类
  - 1、最少硬币问题
  - 2、硬币组合问题
- 最少硬币问题
  - 给定不同面额的硬币和一个总金额，计算可以凑成总金额所需的最少硬币个数。如果没有任何一种硬币组合能组成总金额，返回 -1。假设每一种面额的硬币有无限个。
  - 该问题是一个最优化问题。
- 硬币组合问题
  - 给定不同面额的硬币和一个总金额，计算可以凑成总金额的硬币的组合数。假设每一种面额的硬币有无限个。
  - 该问题求方法总数，不是最优化问题。

最少硬币问题

- 示例 1：
  - 输入：coins = [1, 2, 5], amount = 11
  - 输出：3
  - 解释：11 = 5 + 5 + 1
- 示例 2：
  - 输入：coins = [2], amount = 3
  - 输出：-1
- 思考：如何用回溯法求解该问题

最少硬币问题（续）

- 1、分析最优解的结构，确定dp数组以及下标的含义
  - dp[j]：凑足总额为 j（下标）所需硬币的最少个数
- 2、确定递推公式
  - dp[j] = min(dp[j - coins[i]] + 1, dp[j]); // i 是所有可选硬币的索引
- 3、dp数组如何初始化
  - 首先凑足总金额为0所需钱币的个数一定是0，那么dp[0] = 0;
  - 由于是求最小值，下标非 0 的 dp[] 元素都应该初始化为最大值。
- 4、确定遍历顺序
  - 从 j=0 开始往大递推。
  - 本题求钱币最小个数，那么钱币有顺序和没有顺序都可以，因此钱币和金额的二维遍历先后顺序都可以。
- 5、举例推导dp数组
  - 以输入：coins = [1, 2, 5], amount = 5为例。

最少硬币问题（续）

- 举例推导dp数组
  - 以输入coins = [1, 2, 5], amount = 5为例
  - 注意：dp[j] = min(dp[j - coins[i]] + 1, dp[j])

	dp[0]	dp[1]	dp[2]	dp[3]	dp[4]	dp[5]
加入 1 分硬币	0	1	2	3	4	5
加入 2 分硬币	0	1	1	2	2	3
加入 5 分硬币	0	1	1	2	2	1

最少硬币问题代码

```
class Solution {
public:
    int coinChange(vector<int>& coins, int amount) {
        vector<int> dp(amount + 1, INT_MAX);
        dp[0] = 0;
        for (int i = 0; i < coins.size(); i++)  {              // 遍历物品
            for (int j = coins[i]; j <= amount; j++)  {     // 遍历背包
                if ( dp[j - coins[i]] != INT_MAX )  {       // 如果dp[j -coins[i]]是初始值则跳过
                    dp[j] = min(dp[j - coins[i]] + 1,  dp[j]);
                }
            }
        }
        if (dp[amount] == INT_MAX) return -1;
        return dp[amount];
    }
};
```

硬币组合问题

- 示例 1：
  - 输入: amount = 5, coins = [1, 2, 5]
  - 输出: 4
  - 解释: 有四种方式可以凑成总金额
    - 5=5
    - 5=2+2+1（注：题目是求组合数，因此1+2+2和2+2+1只算一次）
    - 5=2+1+1+1
    - 5=1+1+1+1+1
- 示例 2:
  - 输入: amount = 3, coins = [2]
  - 输出: 0
  - 解释: 只用面额 2 的硬币不能凑成总金额 3

硬币组合问题（续）

1、 确定dp数组以及下标的含义

- dp[j]：凑成总金额 j 的硬币的组合数

2、 确定递推公式

- dp[j] += dp[j - coins[i]]
- 即 dp[j] 就是所有的 dp[j - coins[i]] 相加

3、dp数组如何初始化

- dp[0] = 1，即凑成总金额 0 的货币组合数为 1
- 下标非 0 的 dp[j] 初始化为 0

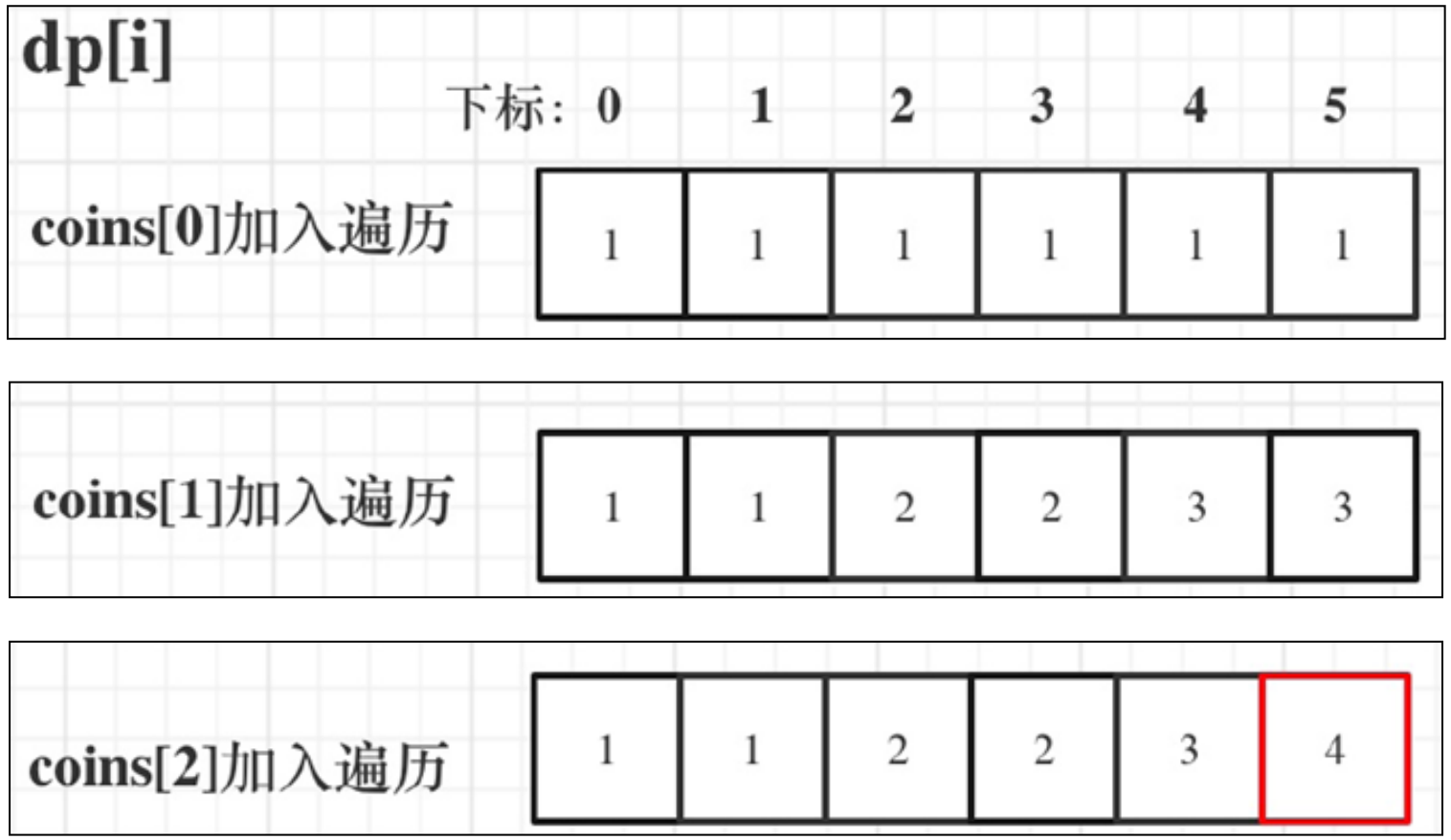
4、确定遍历顺序

- 外层for循环遍历物品（钱币），内层for遍历背包（金钱总额），求出的是组合数。
- 如果反过来，则求出的是排列数。

硬币组合问题（续）

5、举例推导dp数组

- 输入：amount = 5, coins = [1, 2, 5] ，dp 状态图如下：
- 注意：dp[j] += dp[j - coins[i]]



硬币组合问题代码

```
class Solution {
    public:
        int change(int amount, vector<int>& coins) {
            vector<int> dp(amount + 1, 0);
            dp[0] = 1;
            for (int i = 0; i < coins.size(); i++) {              // 遍历物品
                for (int j = coins[i]; j <= amount; j++) {    // 遍历背包
                    dp[j] += dp[j - coins[i]];
                }
            }
            return dp[amount];
        }
    };
```

最大子段和问题的定义

给定由n个整数（可能为负数）组成的序列$a_1,a_2,\ldots,a_n$，求该序列形如 $\sum_{k=i}^{j} a_k$ 的子段和的最大值。当所有整数均为负整数时，定义其最大子段和为0。

即最大字段和为：

$$\max\{0, \max_{1\le i\le j\le n} \sum_{k=i}^{j} a_k\}$$ （目标函数）

例如：当（a1,a2,a3,a4,a5,a6）=(-2,11,-4,13,-5,-2)时，最大子段和为

$$\sum_{k=2}^{4} a_k = 20$$

最大子段和的蛮力法求解

```
int  maxsum(int n, int *a, int & besti,  int & bestj)
{
   int sum=0;
   for (int i=1; i<=n; i++)        //控制子数组开始位置
      for (int j=i; j<=n; j++)      //控制子数组结束位置
      {    int thissum=0;
           for (int k=i; k<=j; k++)  thissum+=a[k];  //子数组循环
           if  (thissum>sum)   {
               sum=thissum;
               besti=i;   bestj=j; //记录位置 }
       }
    return  sum;
 }
```

时间复杂度:

$O(n^3)$

最大子段和的分治法求解

如果将所给的序列 a[1:n] 分为长度相等的两段 a[1:n/2] 和 a[n/2 +1:n]，分别求出这两段的最大子段和，则 a[1:n] 的最大子段和有以下三种可能：

1、a[1:n]的最大子段和与 a[1:n/2] 的最大子段和相同；

2、a[1:n]的最大子段和与 a[n/2+1:n] 的最大子段和相同；

3、a[1:n]的最大子段和为 $\sum_{k=i}^{j} a_k$，1<=i<=n/2, n/2+1<=j<=n

即原问题的解是这三情况中最大的

（a1,a2,a3,a4,a5,a6,a7,a8,a9,a10）

情况1　　情况3　　情况2

对于情况3，在a[1:n/2]计算出 $s1=\max\limits_{1\le i\le n/2}\sum\limits_{k=i}^{n/2} a_k$

在a[n/2+1:n]计算出 $s2=\max\limits_{n/2\le i\le n}\sum\limits_{k=n/2+1}^{n} a_k$

则s1+s2为第3种情形的最优解。

最大子段和的分治法求解（续）

```
int  maxsubsum(int  *a,  int left,  int right)
{  intsum=0;
  if (left==right)  sum=a[left]>0?a[left]:0;
  else
  { int  center=(left+right)/2;
    int leftsum=maxsubsum(a, left, center);
    int rightsum= maxsubsum(a, center+1, right);
    int s1=0;   int lefts=0;
    for (int i=center;  i>=left;  i--)  //分割点向左的最大值
        { lefts+=a[i];    if (lefts>s1)  s1=lefts;  }
    int s2=0;   int rights=0;
    for (int i=center+1; i<=right;  i++) //分割点向右的最大值
        {rights+=a[i];    if (rights>s2)  s2=rights;  }
    sum=s1+s2;
    if (sum<leftsum)  sum=leftsum; //取三者的最大值
    if (sum<rightsum)  sum=rightsum;  }
    return  sum;  }
```

时间复杂度：**O(nlogn)**

$$T(n) = \begin{cases} O(1) & n <= c \\ 2T(n/2) + O(n) & n > c \end{cases}$$

$$T(n) = O(n \log n)$$

最大子段和的动态规划求解

对于上述分治算法的分析中，若记

$$b[j]=\max_{1\le i\le j}\{\sum_{k=i}^{j}a_k\} \qquad 1\le j\le n$$

则所求的最大字段和为（原问题）：

$$\max_{1\le i\le j\le n}\{\sum_{k=i}^{j}a_k\}=\max_{1\le j\le n}\max_{1\le i\le j}\{\sum_{k=i}^{j}a_k\}=\max_{1\le j\le n}b[j]$$

设计思想：

- b[j] 表示以位置 j 为终点的所有子区间中子段和最大的一个子问题。
- 由于 j 为终点的最大子区间包含了位置 j-1，则以 j-1为终点的最大子区间必然包括在其中；
- 如果 b[j-1] >0, 那么显然 b[j] = b[j-1] + a[j]，用之前最大的一个加上 a[j] 即可，因为 b[j] 必须包含 a[j] ；
- 如果 b[j-1]<=0,那么 b[j] = a[j] ,因为是求最大值，前面的负数必然不能使最终解更大。

当b[j-1]>0时，b[j]=b[j-1]+a[j]，否则b[j]=a[j]

由此可得递归方程（状态转移方程）：

b[j]=max{b[j-1]+a[j], a[j]}，1<=j<=n

最大子段和的动态规划求解（续）

例：设（a1,a2,a3,a4,a5,a6）=(-2, 11, -4, 13, -5, -2)，
则动态规划的递推过程为：

1	2	3	4	5	6
−2	11	−4	13	−5	−2
$b_1=-2$	$b_2=11$	$b_3=7$	$b_4=20$	$b_5=15$	$b_6=13$
sum=0	sum=11	sum=11	sum=20	sum=20	sum=20

b_i：以 a[i] 结尾的子串的最大字段和。
sum：原问题的最大字段和。

最大子段和的动态规划求解（续）

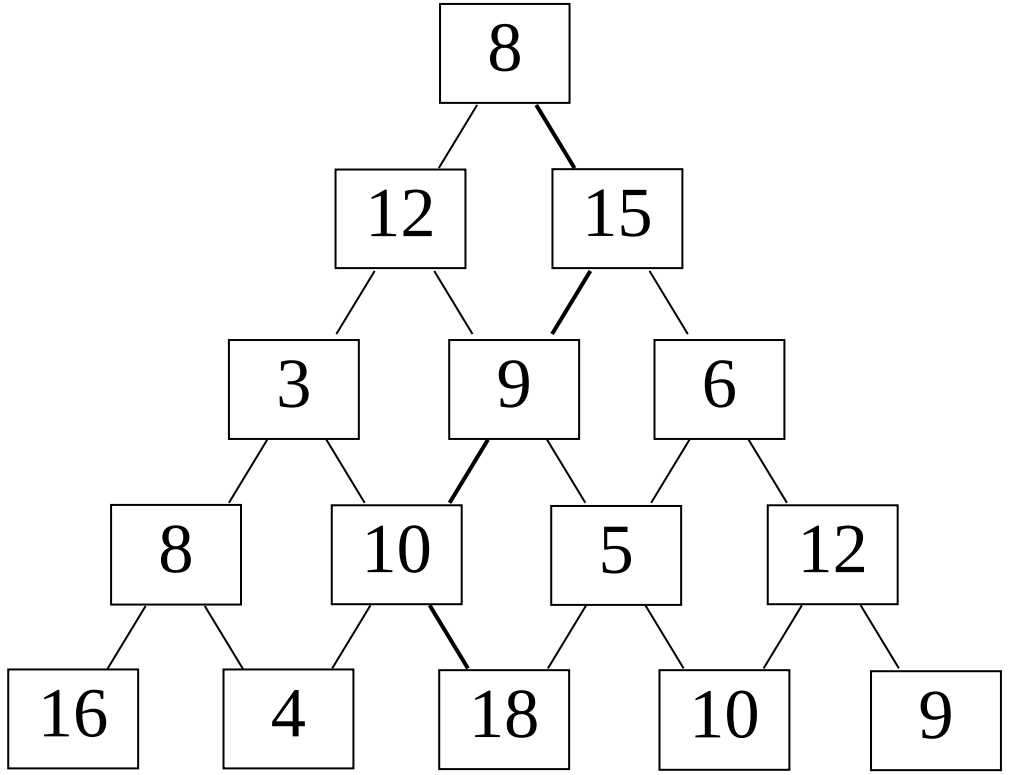
```
int maxsum(int  n,  int *a)  {
    int sum=0, b=0;
    for (int i=1;  i<=n;  i++)     {
        if (b>0) b+=a[i];
        else b=a[i];    //b表示以 i 结尾的子串的最大字段和
        if(b>sum)  sum=b; //sum表示原始问题的最大字段和
     }
     return  sum;    //请思考：如何记录最大子串的起止索引？
}
```

时间复杂度：

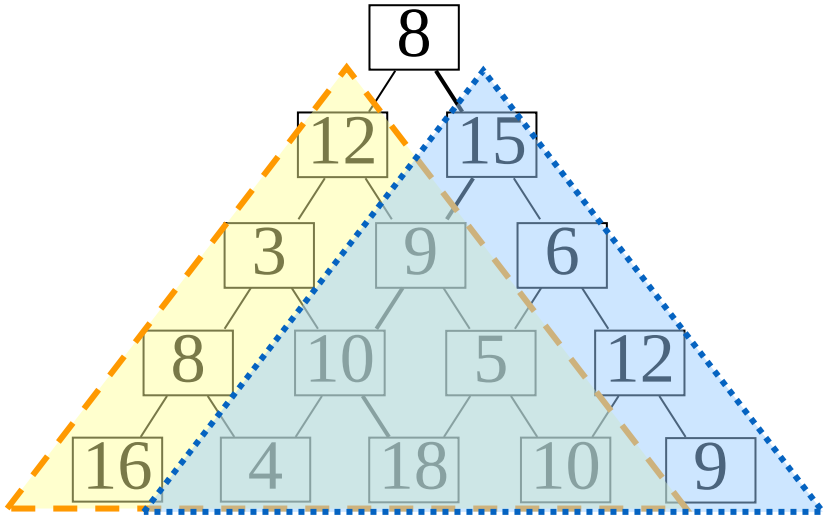
O(n)

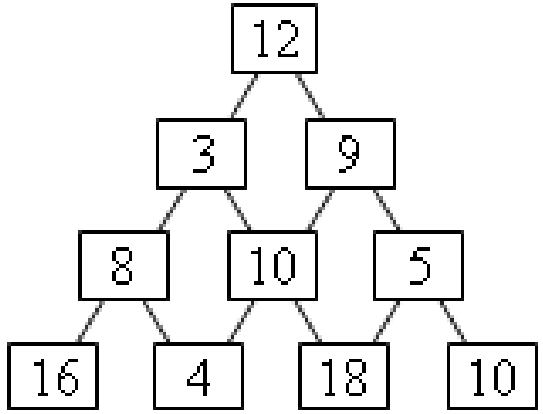
3.2 数塔问题

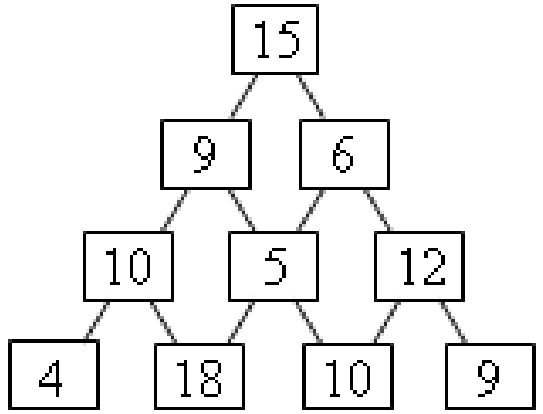
问题描述：从数塔的顶层出发，在每一个结点可以选择向左走或向右走，一直走到最底层，要求找出一条路径，使得路径上的数值和最大。



数塔问题分析







第 1 层的决策	8+max{49,52} = 60				
第 2 层的决策	12+max{31,37} = 49	15+max{37,29} = 52			
第 3 层的决策	3+max{24, 28} = 31	9+max{28,23} = 37	6+max{23,22} = 29		
第 4 层的决策	8+max{16,4} = 24	10+max{4,18} = 28	5+max{18,10} = 23	12+ max{10,9} = 22	
初始化	16	4	18	10	9

自底向上填写

数塔问题分析

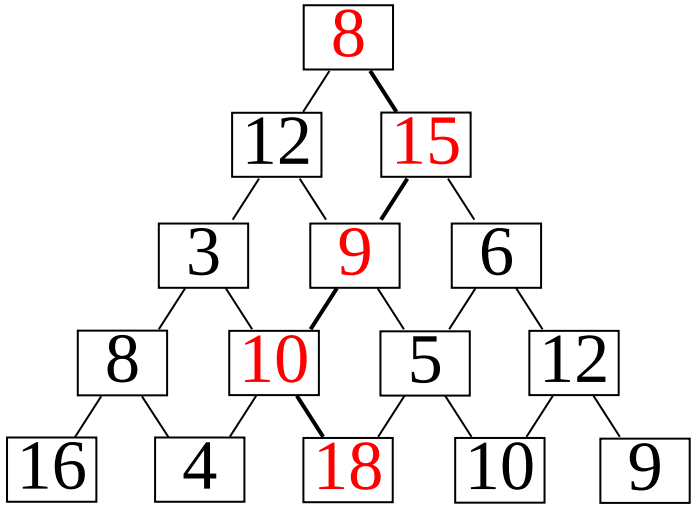
求解初始子问题：最底层的每个数字可以看作 1 层数塔，则最大数值和就是其自身；

求解下一阶段子问题：第 4 层的决策是在底层决策的基础上进行求解，可以看作 4 个 2 层数塔，对每个数塔进行求解；

求解下一阶段子问题：第 3 层的决策是在第 4 层决策的基础上进行求解，可以看作 3 个 3 层的数塔，对每个数塔进行求解；

……

直到最后一个阶段：第 1 层的决策结果就是数塔问题的整体最优解。



数塔问题设计

按三角形的行划分阶段，若行数为 n，则可把问题看做一个n-1个阶段的决策问题。

先求出第 n-1 阶段（第n-1行上各点）到第 n 行的最大和，再依次求出第 n-2 阶段、第 n-3 阶段……第 1 阶段（起始点）各决策点至第n行的最佳路径。

阶段：当前所在塔的层数 i

状态：当前所在某层的位置 j

阶段目标：f[i,j] 表示当前的最优值，为从第 i 阶段中的点 j 至第 n 行的最大的数字和。f[i,j] 的初值为数塔节点的值。

状态转移方程：$f[i][j]=f[i][j]+\max(f[i+1][j],f[i+1][j+1])$

数塔问题实现（自底向上迭代）

```
int dp[105][105];
int main()
{
    int T;
    scanf("%d",&T);
    for(int cas=1;cas<=T;++cas){
        int n;
        scanf("%d",&n);
        for(int i=1;i<=n;i++){
            for(int j=1;j<=i;j++){
                scanf("%d",&dp[i][j]);
            }
        }
        for(int i=n-1;i>=1;i--){
            for(int j=1;j<=i;j++){
                dp[i][j]+=max(dp[i+1][j],dp[i+1][j+1]);
            }
        }
        printf("%d\n",dp[1][1]);
    }
    return 0;
}
```

录入数塔的值

自底向上递推

定义一个二维数组，记录阶段目标

- 阶段目标的初值：数塔本身

倒推进行

- 最高层上，f[i][j] =当前数塔的值
- 上一层：

  f[i][j]=f[i][j]+max(f[i+1][j],f[i+1][j+1])
- 最终目标：f[1][1]

数塔问题实现（递归法）

```
int dp[105][105],a[105][105];
int n;
int cal(int x,int y)
{
    if(dp[x][y]!=-1) return dp[x][y];
    if(x==n) dp[x][y]=a[x][y];
    else dp[x][y]=a[x][y]+max(cal(x+1,y),cal(x+1,y+1));
    return dp[x][y];

}
```

dp是备忘录，数塔本身保存在a数组中

返回备忘录

保存备忘录

```
int main()
{
    int T;
    scanf("%d",&T);
    for(int cas=1;cas<=T;++cas){
        scanf("%d",&n);
        memset(dp,0xff,sizeof(dp));
        for(int i=1;i<=n;i++){
            for(int j=1;j<=i;j++){
                scanf("%d",&a[i][j]);
            }
        }
        printf("%d\n",cal(1,1));
    }
    return 0;
}
```

3.3 矩阵连乘

问题描述: 给定 n 个矩阵 $\{A_1, \ldots, A_n\}$, 其中 A_i 与 A_{i-1} 是可相乘的。确定一个计算次序，使计算矩阵连乘积 $A_1 \ldots A_n$ 所需计算量最少。

例：三个矩阵相乘，有两种计算顺序 (A*B)*C, A*(B*C)。

假设 A 为 10×100 矩阵，B为100×5矩阵，C为5×50矩阵。

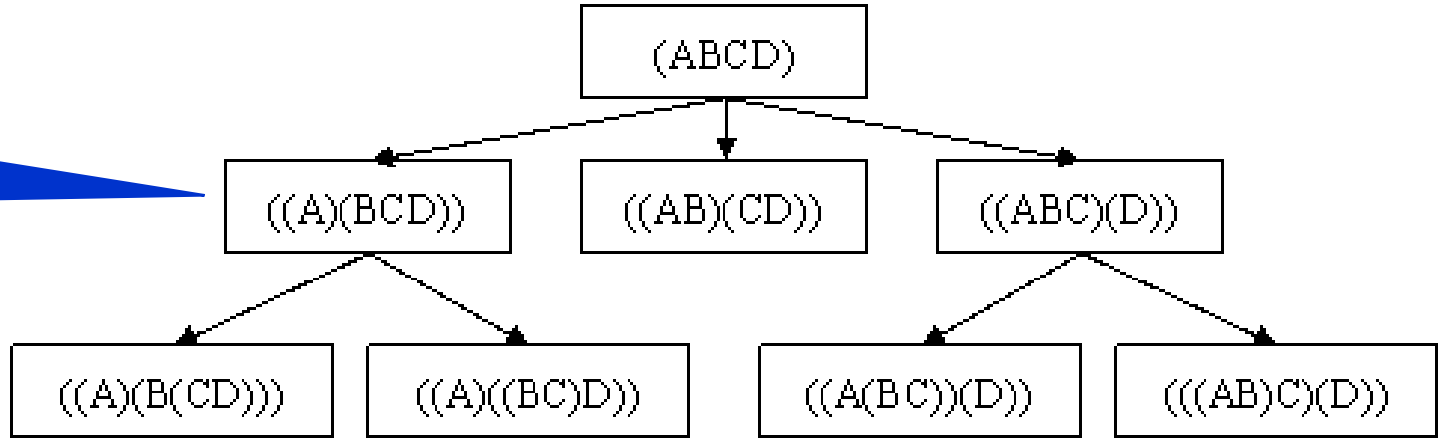
按照顺序1：D=A*B为 10×5 矩阵, 需乘法数为 5000；
D 与 C 乘，所需乘法数为10×5×50=2500, 计算(A*B)*C的总乘法次数为7500。

按照顺序2：E=B*C需乘法数为25000, B*C 为100× 50矩阵；
A 与 E 与乘, 所需乘法数为10×100×50=50000, 计算A*(B*C)的总乘法次数为 75000。

矩阵连乘（续）

- 给定 n 个矩阵 $\{A_1, A_2, \ldots, A_n\}$，其中 A_i 与 A_{i+1} 是可乘的。考察这n个矩阵的连乘积 $A_1A_2\ldots A_n$ 。
- 由于矩阵乘法满足结合律，所以计算矩阵的连乘可以有许多不同的计算次序。这种计算次序可以用“加括号”的方式来确定。
- 若一个矩阵连乘积的计算次序完全确定，也就是说该连乘积已完全加括号，则可以依此次序反复调用 2 个矩阵相乘的标准算法计算出矩阵连乘积。

穷举加括号的每一个可能位置



矩阵连乘的蛮力法

蛮力法：列举出所有可能的计算次序，并计算出每一种计算次序相应需要的数乘次数，从中找出一种数乘次数最少的计算次序。

算法复杂度分析：

对于 n 个矩阵的连乘积，设其不同的计算次序总数为 T(n)。

由于每种加括号方式都可以分解为两个子矩阵的加括号问题：$(A_1 \ldots A_k)(A_{k+1} \ldots A_n)$

可以得到关于 T(n) 的递推式如下：

$$T(n) = \begin{cases} 1 & n = 1 \\ \sum_{k=1}^{n-1} T(k)T(n-k) & n > 1 \end{cases} \Rightarrow T(n) = \Omega(4^n / n^{3/2})$$

K是所有可能的加括号位置

蛮力法求最小矩阵连乘的时间复杂度为指数阶！

矩阵连乘中在某个位置加括号的计算量

将矩阵连乘积 $A_iA_{i+1}\cdots A_j$ ，简记为 **A[i:j]** ，i≤j。

假设在第 k 的位置加括号：

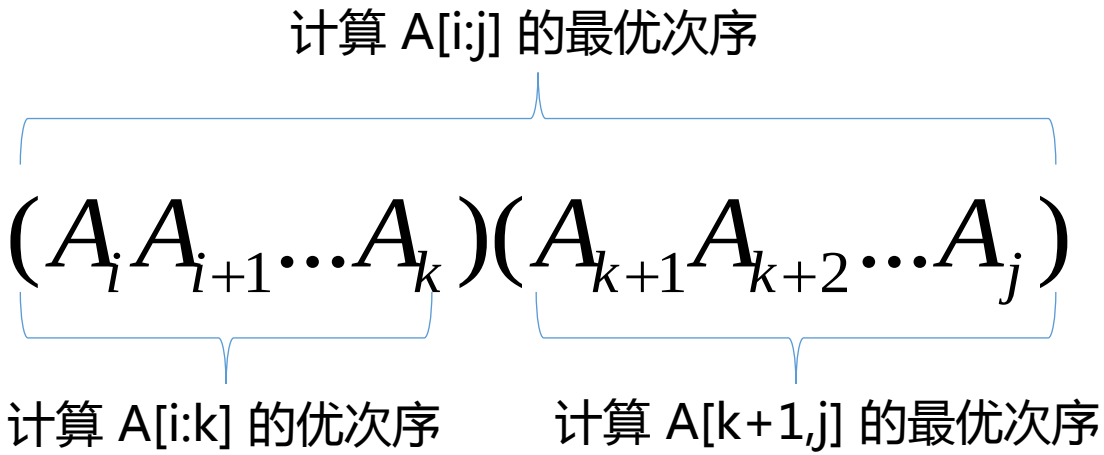
$$(A_iA_{i+1}\cdots A_k)(A_{k+1}A_{k+2}\cdots A_j)$$

则计算 A[i:j] 的总的计算量为：

A[i:k] 的计算量加上 A[k+1:j] 的计算量，

再加上 A[i:k] 和 A[k+1:j] 相乘的计算量

矩阵连乘的动态规划法-最优子结构



分析最优解的结构

- 计算A[i:j]的最优次序所包含的计算矩阵子链 A[i:k]和A[k+1:j] 的次序也是最优的。
- 矩阵连乘计算次序问题的最优解包含着其子问题的最优解，即满足最优子结构性质（问题的最优子结构性质是该问题可用动态规划算法求解的显著特征）。

矩阵连乘的动态规划法-递推方程

矩阵连乘积 $A_iA_{i+1}\cdots A_j$ ，简记为**A[i:j]** （ i≤j ）， A_i 维数为**$P_{i-1}\times P_i$** ，计算 A[i:j] 所需最小数乘次数记为 **m[i][j]。**

- 设计算A[i:j]（ 1≤i≤j≤n ）所需的最少数乘次数 m[i,j]，则原问题的最优解为m[1,n]
- 当 i=j 时，A[i:j]=A_i，因此 m[i,i]=0，i=1,2,...,n （边界条件）
- 当 i<j 时， $m[i,j]=m[i,k]+m[k+1,j]+p_{i-1}p_kp_j$ （针对某个加括号位置 k）
- 因此可以递归地定义 m[i,j] 为（状态转移方程）：

$$m[i,j]=\begin{cases}0 & i=j\\ \min\limits_{i\le k<j}\{m[i,k]+m[k+1,j]+p_{i-1}p_kp_j\} & i<j\end{cases}$$

这种dp叫区间dp

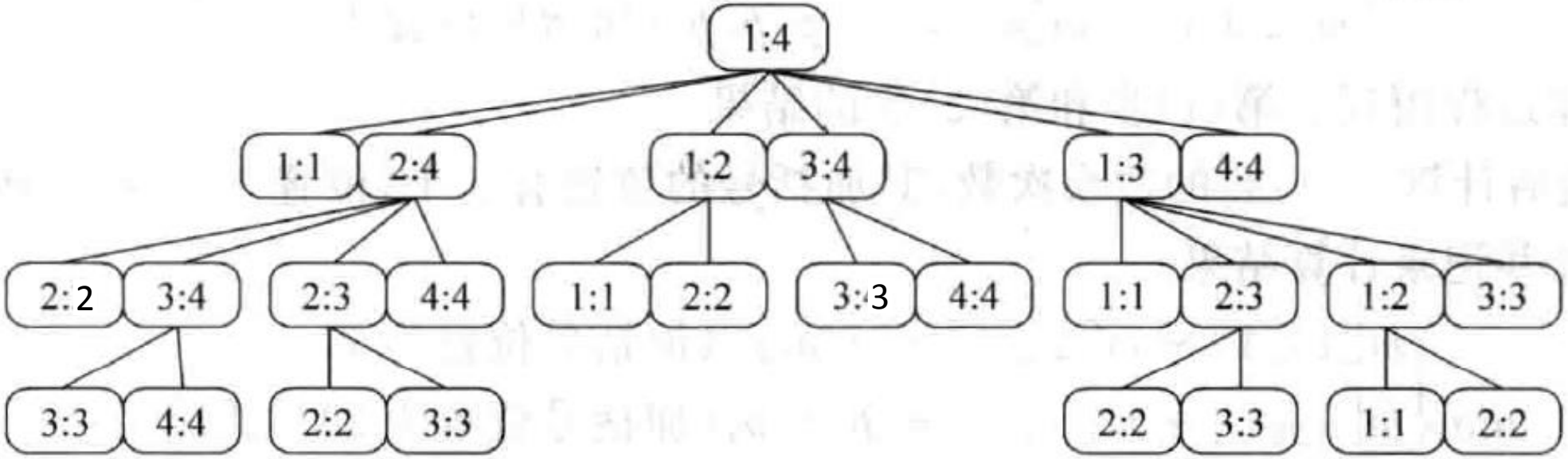
注意：k 的位置只有 j-i 种可能

矩阵连乘的直接递归——算法描述

根据m[i,j]的递推公式，可以得到直接采用递归的算法

```
int recmat(int i , int j)
{
      if (i==j) return 0;
      int u = recmat(i , i) + recmat(i+1 , j) + p[i-1]*p[i]*p[j];
      s[i][j] = i;
      for (int k = i+1; k < j; k++) {   //穷举所有加括号位置
          int t = recmat(i , k) + recmat(k+1 , j) + p[i-1]*p[k]*p[j];
          if (t < u) {
              u = t;            //数乘的最小值
              s[i][j] = k;    //s[i][j]是达到最小值的加括号位置
          }
       }
      return u;
}
```

矩阵连乘的直接递归——递归树



直接递归很多子问题被重复计算

复杂度分析

$$T(n) \geq \begin{cases} O(1) & n=1 \\ \sum_{k=1}^{n-1}(T(k)+T(n-k)+O(1)) & n>1 \end{cases}$$

$$T(n) \geq O(n)+\sum_{k=1}^{n-1}T(k)+\sum_{k=1}^{n-1}T(n-k)=O(n)+2\sum_{k=1}^{n-1}T(k) \qquad T(n) \geq 2^{n-1}$$

矩阵连乘的带备忘录递归—算法描述

通过增加备忘录（m[i,j]）可以得到带备忘录的递归算法

```
int LookupChain(int i，  int j)
{
      if (m[i][j] > 0) return m[i][j]; //m初始值为0，大于0表示已求出子问题解
      if (i == j) return 0;
      int u = LookupChain(i，  i) + LookupChain(i+1，  j) + p[i-1]*p[i]*p[j];
      s[i][j] = i;
      for (int k = i+1; k < j; k++) {     //穷举所有加括号位置
          int t = LookupChain(i，  k) + LookupChain(k+1，  j) + p[i-1]*p[k]*p[j];
          if (t < u) {  u = t;   s[i][j] = k;  } //s[i][j]是达到最小值的加括号位置
        }
      m[i][j] = u;   //将子问题解存入备忘录
      return u;
}
```

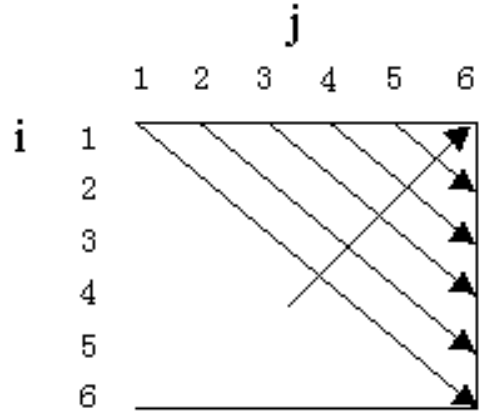
矩阵连乘的递推法——一个例子的递推计算过程

输入的矩阵链

矩阵	A1	A2	A3	A4	A5	A6
数组p	$p_0 \times p_1$	$p_1 \times p_2$	$p_2 \times p_3$	$p_3 \times p_4$	$p_4 \times p_5$	$p_5 \times p_6$
维数值	30×35	35×15	15×5	5×10	10×20	20×25

首先设置6*6的m矩阵（最优值）和s矩阵（分割位置）

递推时必须先计算小的子问题的解，最后得到原问题的解



(a) 计算次序

j

i	1	2	3	4	5	6
1	0	15750	7875	9375	11875	15125
2		0	2625	4375	7125	10500
3			0	750	2500	5375
4				0	1000	3500
5					0	5000
6						0

(b) m[i][j]

j

i	1	2	3	4	5	6
1	0	1	1	3	3	3
2		0	2	3	3	3
3			0	3	3	3
4				0	4	5
5					0	5
6						0

(c) s[i][j]

m[2,5]的计算过程

$$m[2][5] = \min\begin{cases} m[2][2] + m[3][5] + p_1p_2p_5 = 0 + 2500 + 35 \times 15 \times 20 = 13000 \\ m[2][3] + m[4][5] + p_1p_3p_5 = 2625 + 1000 + 35 \times 5 \times 20 = 7125 \\ m[2][4] + m[5][5] + p_1p_4p_5 = 4375 + 0 + 35 \times 10 \times 20 = 11375 \end{cases}$$

m[2][5]=7125，即计算A[2:5]的最少数乘次数； s[2][5]=3，即 A[2:5]最佳分割位置为k=3

矩阵连乘的递推法——一个例子的递推计算过程（续）

注意递推计算 m[i,j] 的顺序是：

m[0,0],m[1,1]…,m[6,6];

m[1,2],m[2,3]…,m[5,6];

m[1,3],m[2,4]…,m[4,6];

m[1,4], m[2,5], m[3,6];

m[1,5], m[2,6];

m[1,6]（原问题）

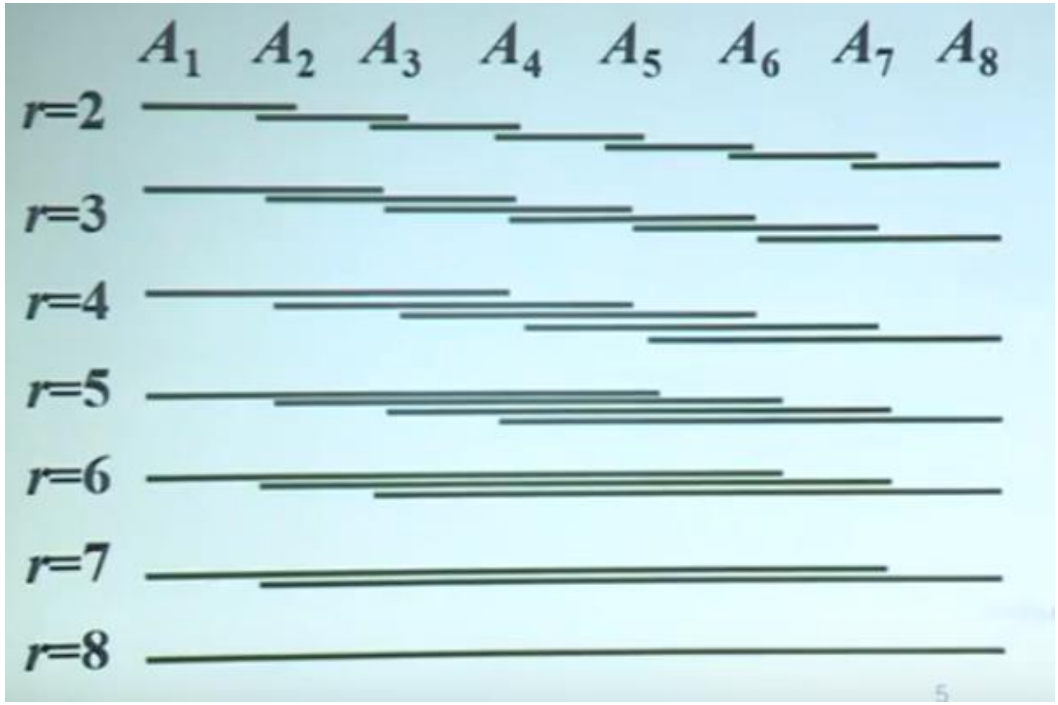
即先计算间隔为0的矩阵链乘法次数，再计算间隔为1的，为2的, …，直到原问题

m[i, j] 的详细计算依据

i \ j	1	2	3	4	5	6
1	0 {A1}	30*35*15 ={A1}A2	A1{A2A3} {A1A2}A3	A1{A2A3A4} {A1A2}{A3A4} {A1A2A3}A4	A1{A2A3A4A5} {A1A2}{A3A4A5} {A1A2A3}{A4A5} {A1A2A3A4}A5	A1{A2A3A4A5A6} {A1A2}{A3A4A5A6} {A1A2A3}{A4A5A6} {A1A2A3A4}{A5A6} {A1A2A3A4A5}A6
2		0 {A2}	35*15*5= {A2}A3	A2{A3A4} {A2A3}A4	A2{A3A4A5} {A2A3}{A4A5} {A2A3A4}A5	A2{A3A4A5A6} {A2A3}{A4A5A6} {A2A3A4}{A5A6} {A2A3A4A5}A6
3			0 {A3}	30*35*15= {A3}A4	A3{A4A5} {A3A4}A5	A3{A4A5A6} {A3A4}{A5A6} {A3A4A5}A6
4				0 {A4}	15*5*10= {A4}A5	A4{A5A6} {A4A5}A6
5					0 {A5}	10*20*25= {A5}A6
6						0 {A6}

矩阵连乘的递推法——一个例子的递推计算过程（续）

- 子链长度为 1 时：初值（1..1，2..2，…，n..n ）
- 子链长度为 2 时：n-1个子问题（1..2，2..3，3..4，…，n-1..n）
- 子链长度为 3 时：n-2个子问题（1..3，2..4，3..5，…，n-2..n）
- …
- 子链长度为 n-1 时：2个子问题（1..n-1， 2..n）
- 子链长度为 n 时：1个子问题（即原问题1..n）



矩阵连乘的递推法—算法描述

```
void MatrixChain(int *p , int n , int **m , int **s)
{
        for (int i = 1; i <= n; i++) m[i][i] = 0;  //初始化m[i][i]
        for (int r = 2; r <= n; r++)   //r 为子链长度
        {
          for (int i = 1; i <= n - r+1; i++)  {  //i：子链开始位置
              int j=i+r-1;   //j：子链结束位置
              m[i][j] = m[i][i] + m[i+1][j]+ p[i-1]*p[i]*p[j]; //初始化：在 i 处加括号
              s[i][j] = i; //初始化：在 i 处加括号
              for (int k = i+1; k < j; k++)    { //穷举所有加括号位置 k, 找最小值
                 int t = m[i][k] + m[k+1][j] + p[i-1]*p[k]*p[j];
                 if (t < m[i][j])   {  m[i][j] = t;   s[i][j] = k;  }
              }
          }
}
```

复杂度分析：matrixChain的主要计算取决于算法中 r，i 和 k 的 3 重循环，循环最内层为O(1)，因此**复杂度为$O(n^3)$**。

矩阵连乘的递推法—小结

迭代计算的关键：

- 每个子问题只计算一次
- 迭代过程
  - 从最小的子问题开始计算
  - 考虑计算顺序，以保证后面用到的值前面已经计算好
  - 利用备忘录存储结构保存计算结果
- 解的追踪
  - 设计标记函数标记每步的决策
  - 考虑根据标记函数追踪解的算法

动态规划算法时间复杂度估算：

- 时间复杂性=每项备忘录计算量之和+追踪解工作量
- 通常追踪工作量不超过计算工作量，是问题规模的多项式函数

3.4 最长公共子序列

子序列：对给定序列 $X=(x_1, x_2,..., x_m)$ 和序列 $Z=(z_1, z_2,..., z_k)$，Z 是 X 的子序列当且仅当存在一个严格递增下标序列 $(i_1, i_2,..., i_k)$，使得对于所有 $j=1, 2, ..., k$，有 $z_j=x_{i_j}$ $(1\le i_j\le m)$。

例如，序列X=(a, b, c, b, d, b)，子序列为Z=(a, c, b, d, b)，下标序列为(1, 3, 4, 5, 6)。

公共子序列：给定两个序列 X和 Y，当另一个序列 Z 既是 X 的子序列又是 Y 的子序列时，称 Z 是序列 X 和 Y 的公共子序列。

最长公共子序列：序列 X 和 Y 的公共子序列中长度最长的那个。

例如：序列X=(a, b, c, b, d, b)，Y=(a, c, b, b, a, b, d, b, b)，它们的最长公共子序列为 Z=(a, c, b, d, b)，在 X 中的递增下标序列是 (1, 3, 4, 5, 6)，在 Y 中的递增下标序列是 (1, 2, 6, 7, 9)。

最长公共子序列—递推函数

要找出序列 $X=\{x_1, x_2,\ldots, x_m\}$ 和 $Y=\{y_1, y_2,\ldots, y_n\}$ 的最长公共子序列，可按下述递推方式计算：

（1）当$x_m=y_n$时，找出 X_{m-1} 和 Y_{n-1} 的最长公共子序列，然后在其尾部加上 x_m 即可得到 X 和 Y 的最长公共子序列；

（2）当$x_m\neq y_n$时，必须求解两个子问题：

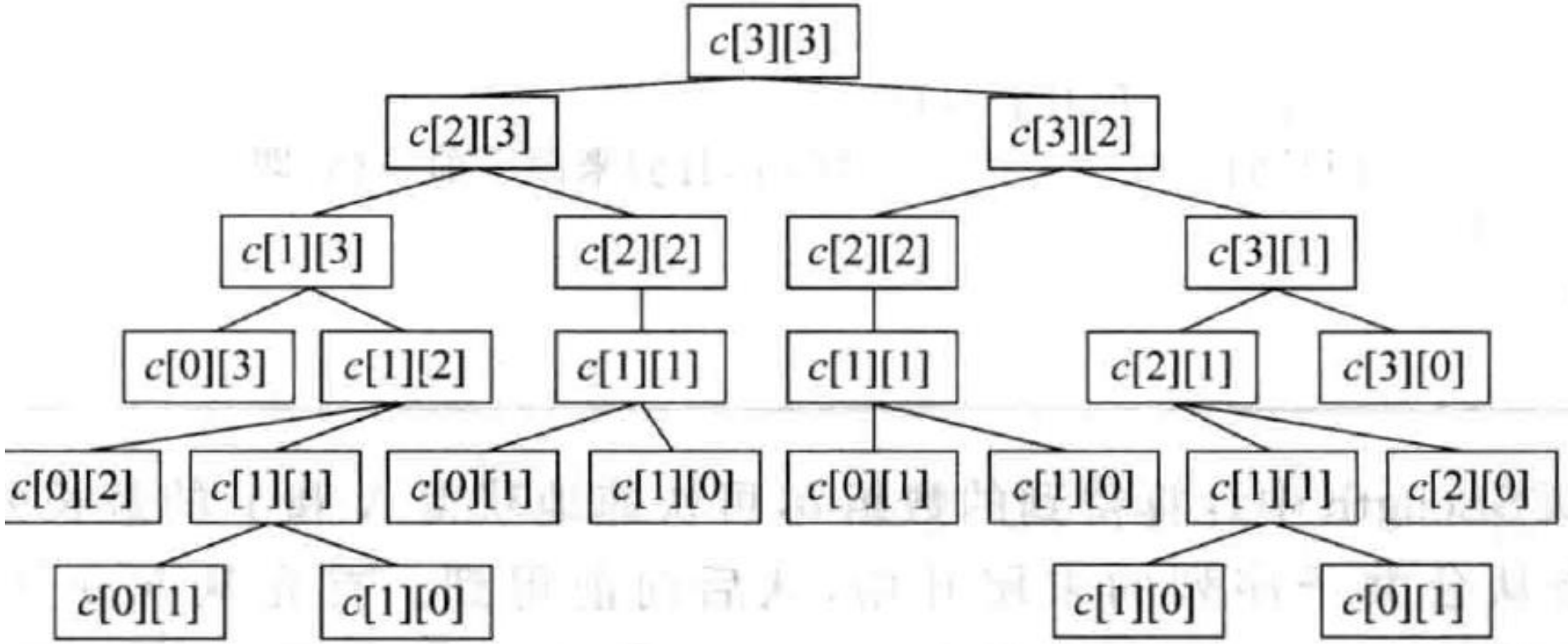
找出 X_{m-1} 和 Y_n 的最长公共子序列，以及 X_m 和 Y_{n-1} 的最长公共子序列，这两个公共子序列中的较长者即为 X 和 Y 的最长公共子序列。

用 L[i][j] 表示子序列 X_i 和 Y_j 的最长公共子序列的长度，可得如下递推函数：

$$L[0][0] = L[i][0] = L[0][j] = 0 \quad (1\le i\le m, 1\le j\le n) \qquad (1)$$

$$L[i][j]=\begin{cases} L[i-1][j-1]+1 & x_i = y_j, i>1, j>1 \\ \max\{ L[i][j-1], L[i-1][j] \} & x_i \neq y_j, i>1, j>1 \end{cases} \qquad (2)$$

最长公共子序列—递归法



计算m=3，n=3序列的递归树

最长公共子序列—递推法

递推法计算计算序列 $X=\{x_1, x_2,\ldots, x_m\}$ 和 $Y=\{y_1, y_2,\ldots, y_n\}$ 的最长公共子序列，分为m 个阶段：

- ✓第1阶段：按照式（2）计算 X_1 和 Y_j 的最长公共子序列长度L[1][j]（$1\le j\le n$），
- ✓第2阶段：按照式（2）计算 X_2 和 Y_j 的最长公共子序列长度L[2][j]（$1\le j\le n$），
- ✓依此类推：最后在第 m 阶段，计算 X_m和 Y_j 的最长公共子序列长度L[m][j]（$1\le j\le n$），则L[m][n]就是序列 X_m 和 Y_n 的最长公共子序列的长度。

最长公共子序列—追踪具体解

为了得到序列 X_m 和 Y_n 具体的最长公共子序列，设二维追踪数组S[m+1][n+1]，其中 S[i][j] 表示在计算 L[i][j] 的过程中的搜索状态，并且有：

$$S[i][j]=\begin{cases}1 & x_i = y_j \\ 2 & x_i \neq y_j \text{且} L[i][j-1] \leq L[i-1][j] \\ 3 & x_i \neq y_j \text{且} L[i][j-1] > L[i-1][j]\end{cases} \quad (3)$$

设计思路：

若S[i][j]=1，表明 $a_i=b_j$，下一个搜索方向是左上角（ S[i-1][j-1] ）

若S[i][j]=2，表明 $a_i \neq b_j$ 且L[i-1][j]≥L[i][j-1]，下一个搜索方向是向上（ S[i-1][j] ）

若S[i][j]=3，表明 $a_i \neq b_j$ 且L[i-1][j] < L[i][j-1]，下一个搜索方向是向左（ S[i][j-1] ）

求解过程：

$X=(x_1,x_2,\ldots,x_7)=(a,b,c,b,d,a,b)$　$Y=(y_1,y_2,\ldots,y_6)=(b,d,c,a,b,a)$

由s的值追溯构造
最长公共子序列

$$L[i][j]=\begin{cases}L[i-1][j-1]+1 & x_i=y_j, i>1, j>1\\ \max\{L[i][j-1],L[i-1][j]\} & x_i\neq y_j, i>1, j>1\end{cases}$$

$$S[i][j]=\begin{cases}1 & x_i=y_j\\ 2 & x_i\neq y_j \text{且} L[i][j-1]\le L[i-1][j]\\ 3 & x_i\neq y_j \text{且} L[i][j-1]>L[i-1][j]\end{cases}$$ (3

		Y		b	d	c	a	b	a
		0	1	2	3	4	5	6
X	0	0	0	0	0	0	0	0
a	1	0	0	0	0	0	1	1
b	2	0	1	1	1	1	1	3
c	3	0	1	1	2	2	2	2
b	4	0	1	1	2	2	1	3
d	5	0	1	1	2	2	2	2
a	6	0	1	1	2	2	2	1
b	7	0	1	2	2	3	4	4

L[i][j]

		Y		b	d	c	a	b	a
		0	1	2	3	4	5	6
X	0	0	0	0	0	0	0	0
a	1	0	0	0	0	0	1	1
b	2	0	1	1	1	1	1	3
c	3	0	2	2	2	1	3	2
b	4	0	1	1	2	2	1	3
d	5	0	2	2	1	2	2	2
a	6	0	2	2	2	2	1	2
b	7	0	1	1	2	2	2	1

S[i][j]

因此最长公共子序列为（x_2,x_3,x_4,x_6)=(b,c,b,a)

s[2][1]=1，因此取X[2]=b=Y[1]

s[3][3]=1，因此取X[3]=c=Y[3]

s[4][5]=1，因此取X[4]=b=Y[5]

最长公共子序列由所有为1的s[i][j]项对应的x_i组成。
此处s[6][6]=1，因此取X[6]=a=Y[6]

显然结果不唯一，(b、d、a、b）也是一个解

算法——最长公共子序列问题

```
int CommonOrder(int m, int n, char x[ ], char y[ ], int [ ][ ]S)
{
  int [ ][ ]L=new int[m+1][n+1];
  for (j=0; j<=n; j++)   //初始化第0行
    L[0][j]=0;
  for (i=0; j<=m; i++)   //初始化第0列
    L[i][0]=0;
  for (i=1; i<=m; i++)                    //逐行向下
    for (j=1; j<=n; j++)                  //从左至右扫描求值
      if (x[i]==y[j])
        {   L[i][j]=L[i−1][j−1]+1;        //由L[i-1][j-1]计算L[i][j]最长公共子序列长度
            S[i][j]=1; // L[i][j] 来自左上角
        }
      else if (L[i-1][j]>=L[i][j-1])
          {   L[i][j]=L[i-1][j];
              S[i][j]=2; // L[i][j] 来自上方
          }
      else {  L[i-1][j]<L[i][j-1];
              S[i][j]=3; // // L[i][j] 来自左方
          }
  return L[m][n];
}
```

S[i][j] 记录 L[i][j] 的值是由三个子问题 L[i-1][j-1]+1，L[i-1][j] 和 L[i][j-1] 中的哪一个计算得到的。
这一信息将用于构造最长公共子序列。

最长公共子序列—时间复杂性

递推法的时间复杂度

✓第一个for循环的时间性能是O(n)；

✓第二个for循环的时间性能是O(m)；

✓第三个循环是两层嵌套的for循环，其时间性能是O(m×n)；

所以，算法的时间复杂性是O(m×n)。

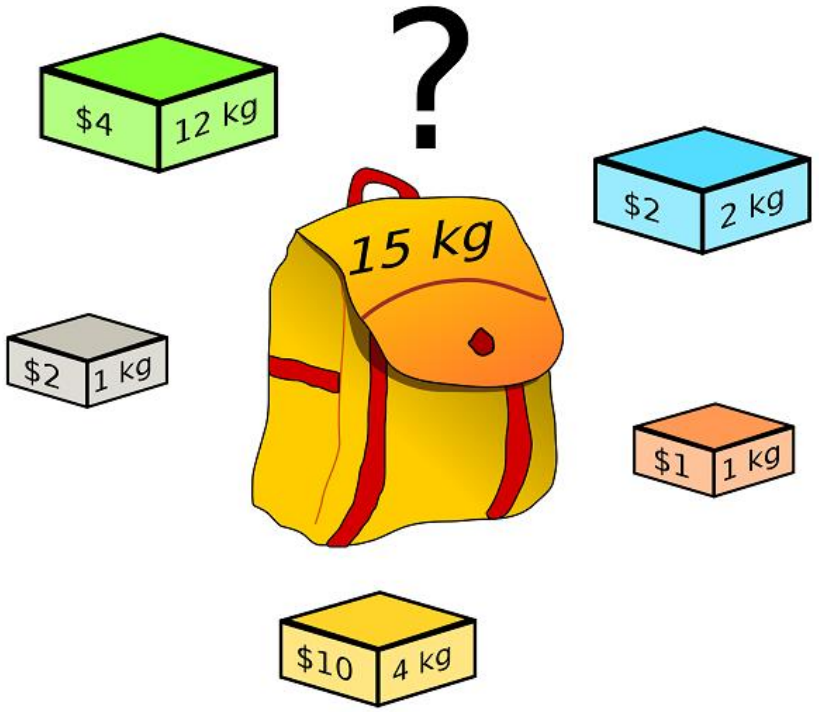
3.5 0/1背包问题（0/1 Knapsack Problem）

什么是背包问题？

给定 n 种物品和一个背包，物品 i 的重量是 w_i，其价值为 v_i，背包容量为 C。背包问题是如何选择装入背包的物品，使得装入背包中物品的总价值最大？

何为0/1？

如果在选择装入背包的物品时，对每种物品 i 只有两种选择：装入背包或不装入背包，即不能将物品 i 装入背包多次，也不能只装入物品 i 的一部分，则称为 0/1背包问题。



背包问题约束条件和目标函数

- 在0/1背包问题中，物品 i 或者被装入背包，或者不被装入背包。
- 设 $(x_1, x_2, \ldots, x_n)$ 是n个物品0/1背包问题的解， x_i 表示物品 i 装入背包的情况：当 $x_i=0$ 时，表示物品 i 没有被装入背包；$x_i=1$ 时，表示物品 i 被装入背包。
- 根据问题的要求，有如下约束条件和目标函数：

约束条件：

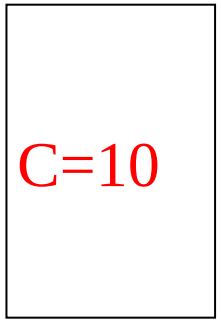
$$\begin{cases} \sum_{i=1}^{n} w_i x_i \le C \\ x_i \in \{0,1\} \quad (1 \le i \le n) \end{cases} \qquad \text{（式1）}$$

目标函数：

$$\max \sum_{i=1}^{n} v_i x_i \qquad \text{（式2）}$$

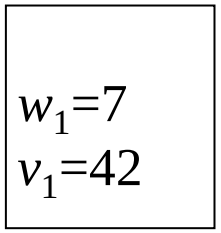
于是，问题归结为寻找一个满足约束条件1，并使目标函数 2 达到最大的解向量$X=(x_1, x_2, \ldots, x_n)$。

回顾：蛮力法解0/1背包问题例子

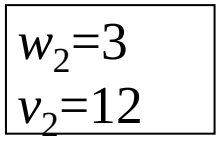


背包

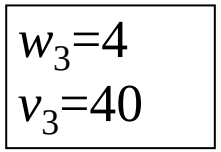
4个物品的重量和价值



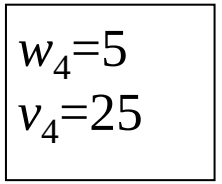
物品1



物品2



物品3



物品4

序号	子集	总重量	总价值	序号	子集	总重量	总价值
1	φ	**0**	**0**	9	{2,3}	**7**	**52**
2	{1}	**7**	**42**	10	{2,4}	**8**	**37**
3	{2}	**3**	**12**	11	{3,4}	**9**	***65***
4	{3}	**4**	**40**	12	{1,2,3}	14	不可行
5	{4}	**5**	**25**	13	{1,2,4}	15	不可行
6	{1,2}	**10**	**54**	14	{1,3,4}	16	不可行
7	{1,3}	11	不可行	15	{2,3,4}	12	不可行
8	{1,4}	12	不可行	16	{1,2,3,4}	19	不可行

$O(2^n)$

背包问题的最优化分析

- 0/1背包问题是否满足最优子结构？

$$\begin{cases} \sum_{i=1}^{n} w_i x_i \le C \\ x_i \in \{0,1\} \quad (1 \le i \le n) \end{cases} \qquad 求 \max \sum_{i=1}^{n} v_i x_i$$

思路：若能证明(x_2，…，x_n)是下面子问题的最优解，则满足。

$$\begin{cases} \sum_{i=2}^{n} w_i x_i \le C - w_1 x_1 \\ x_i \in \{0,1\} \quad (2 \le i \le n) \end{cases} \qquad \max \sum_{i=2}^{n} v_i x_i$$

用反证法可以证明，0/1背包问题具有最优子结构性质，可以采用动态规划方法。

回顾：动态规划的关键问题

- 子问题边界确定
- 如何将原问题归结为更小的子问题
- 确定递推方程和初值
- 确定计算顺序（从小到大的递推顺序）
- 是否需要标记函数以追踪问题的解
- 时间复杂度分析

背包问题的动态规划决策方法

- 0/1背包问题可以看作是决策一个序列 $(x_1, x_2, \ldots, x_n)$，对任一变量 x_i 的决策是决定 $x_i=1$ 还是 $x_i=0$。
- 在对 x_{i-1} 决策后，已确定了$(x_1, \ldots, x_{i-1})$，在决策 x_i 时，问题处于下列两种状态之一：

  （1）背包容量不足以装入物品 i，则 $x_i=0$，背包不增加价值；

  （2）背包容量可以装入物品 i。

- 在（2）的状态下，物品 i 又有两种选择：

  装入（$x_i=1$）或不装入（$x_i=0$）。

  这两种情况下背包价值的最大者是对 x_i 决策后背包的最大价值。

背包问题的递推函数（状态转移函数）

令 V(i, j) 表示在前 i (1≤i≤n) 个物品中能够装入容量为 j (1≤j≤C) 的背包的物品的价值最大值，则可以得到如下递推函数：

$$V(i, 0) = V(0, j) = 0 \qquad (1)\ （边界条件）$$

$$V(i, j) = \begin{cases} V(i-1, j) & j < w_i \\ \max\{V(i-1, j),\ V(i-1, j-w_i) + v_i\} & j \geq w_i \end{cases} \qquad (2)$$

w_i 是第 i 个物品的体积，v_i 是第 i 个物品的价值

当 j 能容纳 w_i 时，要么不装入第 i 个物品，要么装入第 i 个物品，取这两种情况下价值的最大值，作为 (i,j) 下的最优解

背包问题的递推函数（状态转移函数）（续）

- 式(1)表明：把前面 i 个物品装入容量为 0 的背包，或把 0 个物品装入容量为 j 的背包，得到的价值均为 0。
- 式(2)的第一个式子表明：如果第 i 个物品的重量大于背包的容量 j，则物品 i 无法装入背包，因此装入前 i 个物品得到的最大价值和装入前 i-1 个物品得到的最大价值是相同的。
- 式(2)的第二个式子表明：如果第 i 个物品的重量小于背包的容量，则会有以下两种选择：（1）如果第 i 个物品没有装入背包，则背包中物品的价值就等于把前 i-1 个物品装入容量为 j 的背包中所取得的价值。（2）如果把第 i 个物品装入背包，则背包中物品的价值等于把前 i-1 个物品装入容量为 $j-w_i$ 的背包中的价值加上第 i 个物品的价值 v_i。

  取二者中价值较大者作为把前 i 个物品装入容量为 j 的背包中的最优解。

背包问题实例（递推过程和最优解追踪）

实例：有 5 个物品，其重量分别是 {2, 2, 6, 5, 4}，价值分别为{6, 3, 5, 4, 6}，背包的容量为10。

求该背包的最大价值。

	物品1	物品2	物品3	物品4	物品5
重量	2	2	6	5	4
价值	6	3	5	4	6

背包问题实例（递推过程和最优解追踪）（续）

根据背包问题的递推函数，设置一个(n+1)×(C+1)的二维表 V，其中 V[i][j] 表示把前 i 个物品装入容量为 j 的背包中获得的最大价值。

$$V(i,j)=\begin{cases} V(i-1,j) & j<w_i \\ \max\{V(i-1,j),\ V(i-1,j-w_i)+v_i\} & j\geq w_i \end{cases}$$

初值：V(i, 0)= V(0, j)=0

物品 i \ 容量 j	0	1	2	3	4	5	6	7	8	9	10
	0	0	0	0	0	0	0	0	0	0	0
w_1=2 v_1=6	1	0	0	6	6	6	6	6	6	6	6
w_2=2 v_2=3	2	0	0	6	6	9	9	9	9	9	9
w_3=6 v_3=5	3	0	0	6	6	9	9	9	11	11	14
w_4=5 v_4=4	4	0	0	6	6	9	9	10	11	13	14
w_5=4 v_5=6	5	0	0	6	6	9	9	12	12	15	15

x_1=1
x_2=1
x_3=0
x_4=0
x_5=1

最优解是把物品 1、2、5 装入背包

背包问题另一个例子（课堂练习）

练习：填表

i（物品编号）	1	2	3	4
w（体积）	2	3	4	5
v（价值）	3	4	5	6

背包容量 C=8

$$V(i,j)=\begin{cases} V(i-1,j) & j<w_i \\ \max\{V(i-1,j),\ V(i-1,j-w_i)+v_i\} & j\geq w_i \end{cases}$$

i/j	0	1	2	3	4	5	6	7	8
0	0	0	0	0	0	0	0	0	0
1	0								
2	0								
3	0								
4	0								

(背包容量)

(物品)

背包容量为 j 时装入 i 个物品的最大价值

背包问题另一个例子（课堂练习）（续）

i（物品编号）	1	2	3	4
w（体积）	2	3	4	5
v（价值）	3	4	5	6

背包容量 C=8

i/j	0	1	2	3	4	5	6	7	8
0	0	0	0	0	0	0	0	0	0
1	0	0	3	3	3	3	3	3	3
2	0	0	3	4	4	7	7	7	7
3	0	0	3	4	5	7	8	9	9
4	0	0	3	4	5	7	8	9	**10**

背包问题的算法描述

- 设 n 个物品的重量存储在数组 w[n] 中，价值存储在数组 v[n] 中，背包容量为 C，数组 V[n+1][C+1] 存放迭代结果，其中 V[i][j] 表示前 i 个物品装入容量为 j 的背包中获得的最大价值，数组 x[n] 存储装入背包的物品。
- 求解0/1背包问题的动态规划法算法（递推算法）如下：

算法——0/1背包问题

```
int KnapSack(int n, int w[ ], int v[ ]) {
    for (i=0; i<=n; i++)   //初始化第0列
        V[i][0]=0;
    for (j=0; j<=C; j++)   //初始化第0行
        V[0][j]=0;
```

背包问题的算法描述（续）

算法描述——0/1背包问题

```
    for (i=1; i<=n; i++)     //计算第 i 行，进行第 i 次迭代
          for (j=1; j<=C; j++)
                if (j<w[i])   V[i][j]=V[i-1][j];
                else   V[i][j]=max(V[i-1][j], V[i-1][j-w[i]]+v[i]);

     j=C;
     for (i=n; i>0; i--){    //求装入背包的物品
         if (V[i][j]>V[i-1][j]) {
              x[i]=1;   // 第 i 个物品装入
              j=j-w[i];
         }
         else x[i]=0; // 第 i 个物品没有装入
     }
     return V[n][C];    //背包的最大价值
}
```

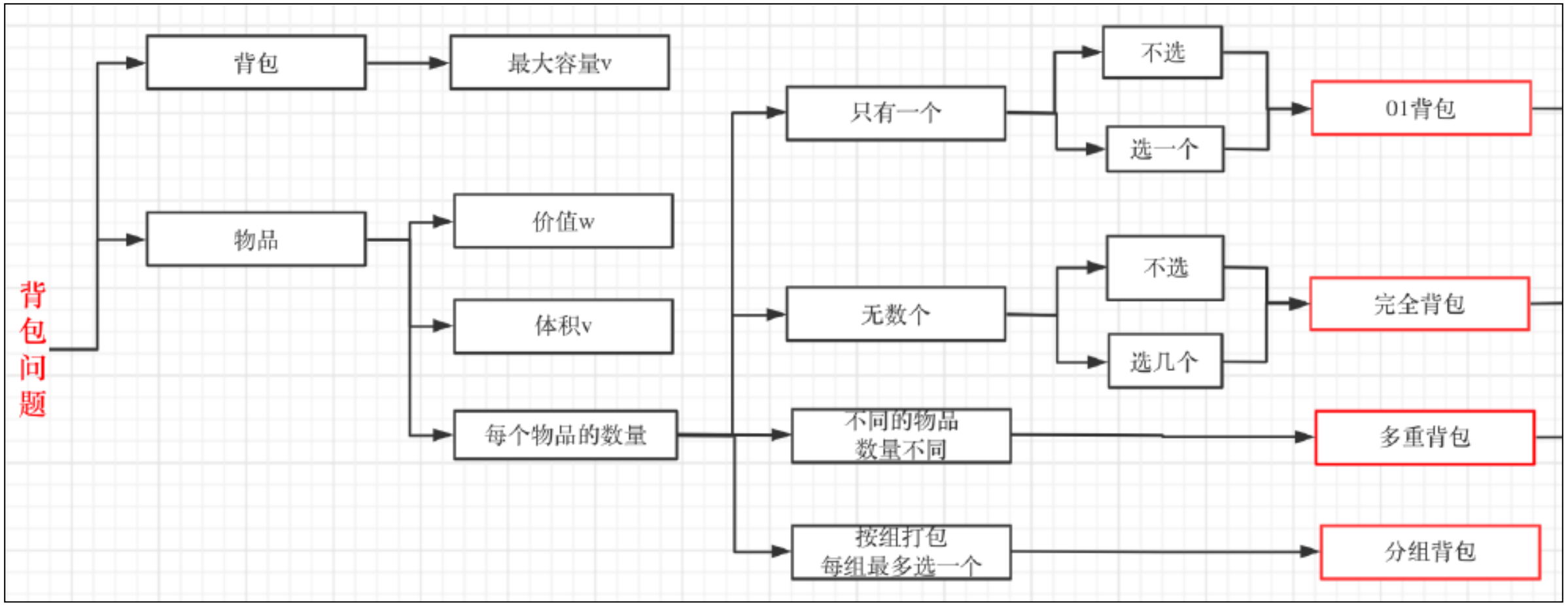
注意如下问题：

- 背包问题的子问题边界的界定
- 列出优化函数的递推方程及边界条件
- 自底向上计算，设计备忘录（表格）
- 标记函数的设立与解的追踪

背包问题的复杂性分析

- 动态规划：
  - 第一个for循环的时间性能是O(n)；
  - 第二个for循环的时间性能是O(C)；
  - 第三个循环是两层嵌套的 for 循环，其时间性能是O(n×C)；
  - 第四个for循环的时间性能是O(n)；
  - 整个算法的时间复杂性为O(n×C)，并且一定能求得最优解。
- 穷举法：$O(2^n)$，一定能求得最优解。
- 贪心法：O(n)，不一定能求得最优解。

背包问题小结



- ✓ 前面讲的零钱兑换问题，实际上是完全背包（每种钱币可以无限提供）。
- ✓ 此外利用滚动数组，可以把背包问题从二维dp降为一维dp。前面讲的零钱兑换已经采用了滚动数组。

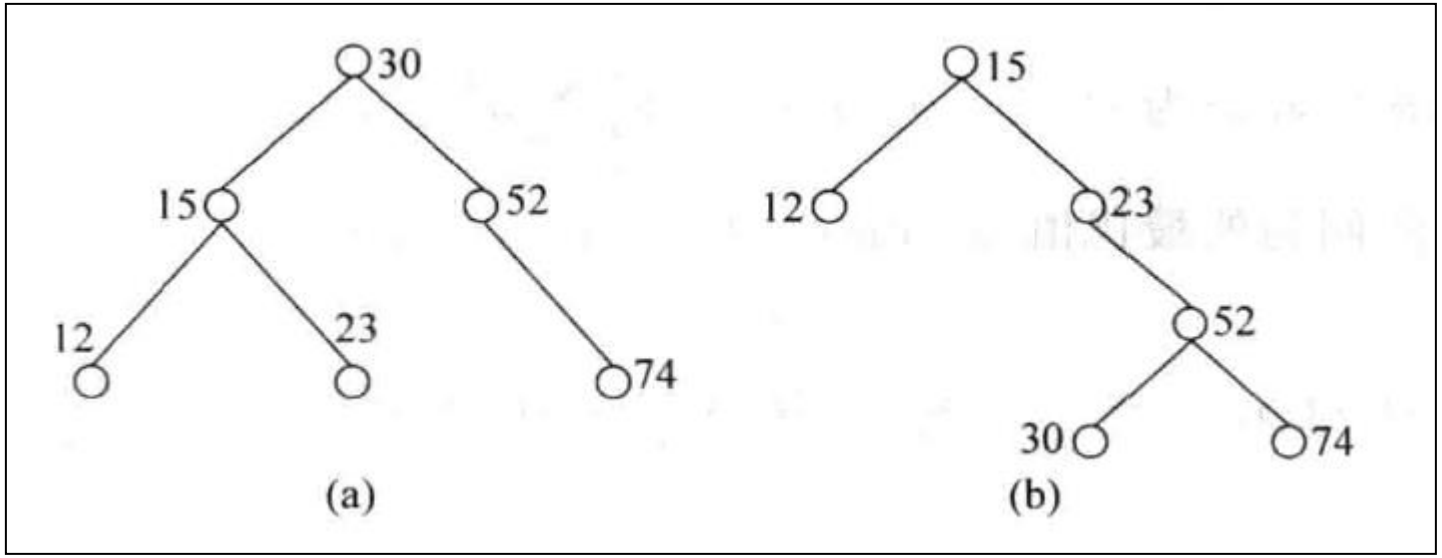
背包问题小结：0/1背包和完全背包

- 递推公式1（二维形式，i 表示物品，j 表示背包容量）
  - 0/1背包：dp[i][j] =max(dp[i-1][j], dp[**i-1**][j-weight[i]]+value[i])
  - 完全背包： dp[i][j] =max(dp[i-1][j], dp[**i**][j-weight[i]]+value[i])
- 递推公式2（采用一维滚动数组，j 表示背包容量）
  - 0/1背包：dp[j] =max(dp[j], dp[j-weight[i]]+value[i])，注意：遍历时 j 必须按照从大到小的顺序。
  - 完全背包： dp[j] =max(dp[j], dp[j-weight[i]]+value[i])，注意：遍历时 j 必须按照 从小到大的顺序。
- 前面的最少硬币问题实际是完全背包问题
  - 要找的总金额是背包容量，每种硬币的重量是coins[i]（硬币可以无限提供），每种硬币的价值都是 1。
  - 此外要求背包必须刚好被装满，否则无解。

3.6 最优二叉查找树

- 二叉查找树（BST—Binary Search Trees）是一棵空树或者满足以下性质的一棵树：
  - 1、每个节点作为搜索对象，它的关键字互不相同。
  - 2、对于树上的所有节点，如果它有左子树，那么左子树上所有节点的关键字都小于该节点的关键字。
  - 3、对于树上的所有节点，如果它有右子树，那么右子树上所有节点的关键字都大于该节点的关键字。

对于一个有序的关键字集合，可以构造其二叉查找树（可能不唯一）



关键字{12，15，23，30，52，74}的两个二叉查找树

二叉查找树的搜索过程

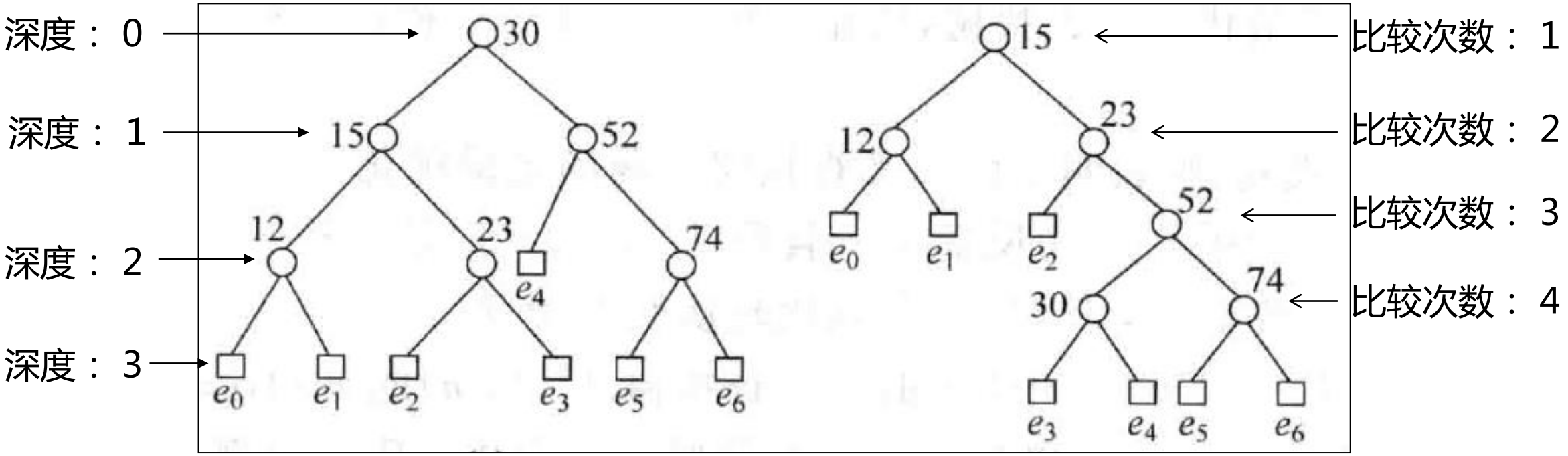
二叉查找树的搜索过程

- 从根节点开始，如果根为空，则搜索不成功，否则使用待搜索值与根节点比较；
- 如果待搜索值等于根节点的关键字，则搜索成功返回；
- 如果小于根节点的关键字，则向左子树搜索；
- 如果大于根节点的关键字，则向右子树搜索。

扩充二叉查找树

- 考虑到搜索不成功的情况，对原始的二叉查找树进行扩充。
- 即在原二叉查找树中，对于原来二叉树里度数为 1 的分支节点，在它下面增加一个空树叶；对于原来二叉树的树叶，在它下面增加两个空树叶。每个空树叶代表一个“查找不成功”的区间。
- 新增加的空树叶（称为“外部节点”）的个数等于原来二叉树的节点（称为“内部节点”）的个数加 1。
- 如果将原二叉查找树的根节点看做是 0 层，则增加外部节点后树的深度增加 1。

扩充后的二叉查找树



- 外部节点 e0 到 e7 表示搜索不到的情形，其查找范围（区间）分别是

  (负无穷,12)，(12,15), (15, 23), (23,30)，(30,52)，(52, 74)，(74,正无穷)

- 查找时，内部节点的“比较次数”为内部节点在二叉树中的深度+1，外部节点的比较次数就为外部节点在二叉树中的深度。

二叉查找树的总比较次数

- 对有 n 个关键字的二叉查找树，设内部节点 x_i 的深度为 c_i，外部节点（x_j, x_{j+1}）的深度为 d_j，则其总的比较次数为：

$$p = \sum_{i=1}^{n}(1+c_i) + \sum_{j=0}^{n} d_j$$

外部节点有 n+1个

例如上页两个二叉查找树的总比较次数分别为：

$$p_a = \sum_{i=1}^{n}(1+c_i) + \sum_{j=0}^{n} d_j = (1+2\times 2+3\times 3)+(1\times 2+6\times 3) = 34$$

$$p_b = \sum_{i=1}^{n}(1+c_i) + \sum_{j=0}^{n} d_j = (1+2\times 2+3\times 1+4\times 2)+(2\times 3+4\times 4) = 38$$

内部节点比较次数　　外部节点比较次数

最优二叉查找树问题描述

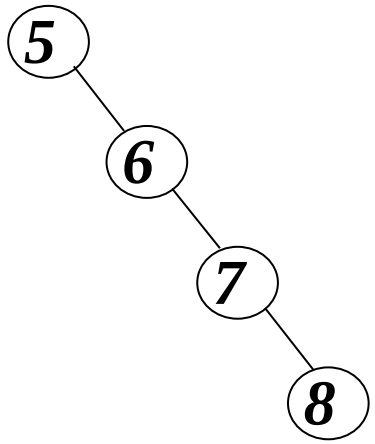
- 假设对内部节点 x_i 的查找概率为 b_i，对外部节点 (x_j, x_{j+1}) 的查找概率为 a_j，则在二叉查找树中进行搜索的平均比较次数为：

$$p = \sum_{i=1}^{n} b_i(1 + c_i) + \sum_{j=0}^{n} a_j d_j \quad \text{且} \quad \sum_{i=1}^{n} b_i + \sum_{j=0}^{n} a_j = 1$$

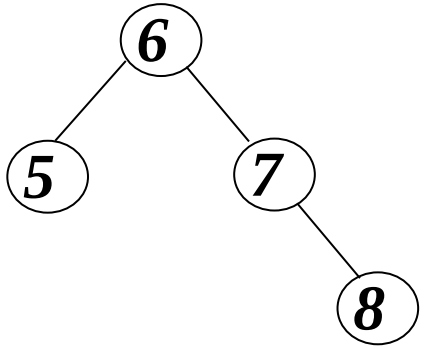
- 最优二叉查找树问题：
  对有序集 S 及其查找概率分布（$a_0, b_1, a_1, b_2, a_2, \ldots, b_n, a_n$），在所有表示有序集 S 的二叉查找树中找出一棵具有最小平均比较次数（min(p)）的二叉查找树。

平均比较次数实例

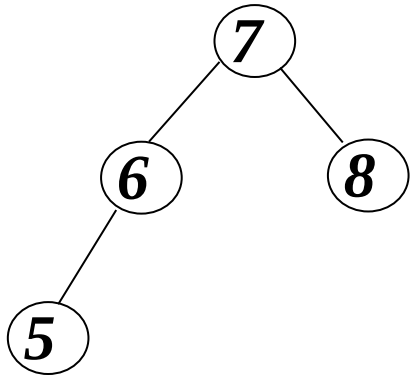
- 例如，设关键字集合 {5, 6, 7, 8} 的查找概率是 {0.1, 0.2, 0.4, 0.3}：
  - 树(a)的平均比较次数是 0.1×1＋0.2×2+0.4×3+0.3×4=2.9；
  - 树(b)的平均比较次数是 0.1×2＋0.2×1+0.4×2+0.3×3=2.1；
  - 树(c)的平均比较次数是 0.1×3＋0.2×2+0.4×1+0.3×2=1.7。



(a)



(b)

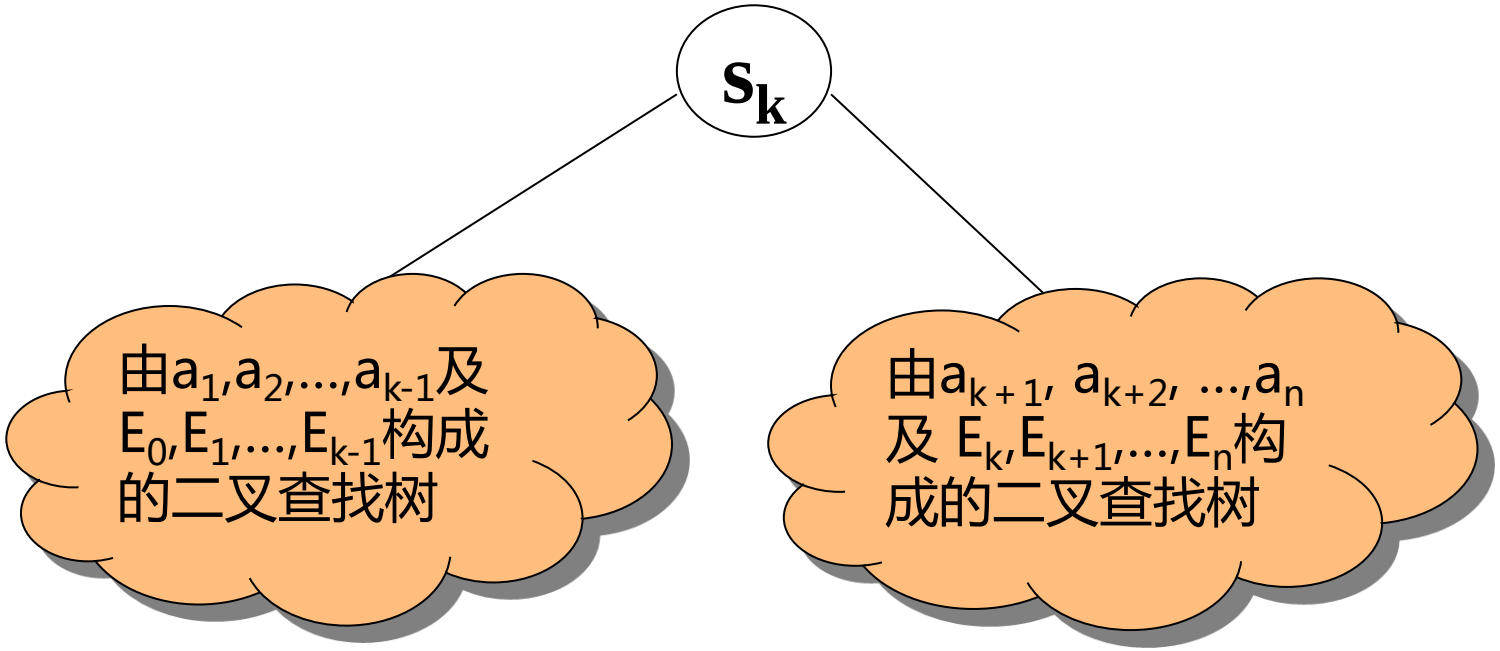


(c)

显然，对于同一关键字集合，不同的二叉查找树的平均比较次数可能是不相同的。

最优二叉查找树的最优化分析

- 设二叉查找树为 T(1, n)，有 n 个内部节点和 n+1 个外部节点。
- 取 s_k（1<=k<=n）作为该树的根节点，则 T(1,n) 的左子树记为 T(1,k-1)，右子树记为 T(k+1, n)。
- 可以证明：如果 T(1,n) 是最优二叉查找树，则左子树 T(1, k-1) 和右子树T(k+1, n)也是最优二叉查找树。



最优二叉查找树的子问题划分

- 设 T(i,j) 是 T(1,n) 的一棵由内部节点 s_i, …, s_j 和外部节点 e_{i-1}, e_i, …,e_j 构成的子树。
- 显然该子树的节点被搜索到的概率总和 w_{ij} 为：

$$w_{ij} = \sum_{m=i}^{j} b_m + \sum_{t=i-1}^{j} a_t = a_{i-1} + b_i + \cdots + b_j + a_j$$

- 可以把 T(i,j) 作为一棵二叉查找树，其各节点的概率分布为：

$$(\bar{a}_{i-1}, \bar{b}_i, \bar{a}_i, \cdots, \bar{b}_j, \bar{a}_j)$$

其中

$$\bar{b}_k = b_k / w_{ij} \quad (i \leqslant k \leqslant j)$$

$$\bar{a}_h = a_h / w_{ij} \quad (i-1 \leqslant h \leqslant j)$$

$$\sum_{m=i}^{j} \bar{b}_m + \sum_{t=i-1}^{j} \bar{a}_t = \bar{a}_{i-1} + \bar{b}_i + \bar{a}_i + \cdots + \bar{b}_j + \bar{a}_j = 1$$

最优二叉查找树的递推方程

- 设 T(i,j) 的平均比较次数为 p_{ij}，当前选择的根节点为 x_m，其左子树 T_l 和 右子树 T_r 的平均比较次数分别为 p_l 和 p_r。则：

$$\begin{aligned} w_{ij}p_{ij} &= w_{i,\,m-1}(p_l+1)+w_{m,m}+w_{m+1,\,j}(p_r+1) \\ &= w_{ij}+w_{i,\,m-1}p_l+w_{m+1,\,j}p_r \quad i\leqslant j \end{aligned}$$

T_l和T_r的比较次数需要加 1

注：$w_{ij} = w_{i,m-1} + w_{m,m} + w_{m+1,j}$

由于最优二叉查找树问题满足最优化原理，因此计算最优 p_{ij} 的递归式如下：

$$w_{ij}p_{ij} = w_{ij} + \min_{i\leqslant k\leqslant j}\{w_{i,k-1}p_{i,k-1} + w_{k+1,j}p_{k+1,j}\} \quad i\leqslant j$$

在所有可能的根节点中选择比较次数最小的

最优二叉查找树的递推方程（续）

- 设 $w_{ij}p_{ij}$ 为 m[i,j]（节点 i 到 j 在原序列下的最小平均比较次数），可以得到计算 m[i,j] 的递推方程为：

$$\begin{cases} m(i,j) = w_{ij} + \min\limits_{i \leqslant k \leqslant j} \{m(i,k-1) + m(k+1,j)\} & i \leqslant j \\ m(i,i-1) = 0 & 1 \leqslant i \leqslant n \end{cases}$$

注意：w_{ij}为节点 i 到 j 的概率总和

当选择第一个（或最后一个）节点作为根节点时，其左子树（或右子树）为空，比较次数为 0

该递推方程和矩阵连乘问题的递推方程很相似

最优二叉查找树的递推算法

- 输入：有序集 $S=\{s_1, s_2, \ldots, s_n\}$，及其概率分布 $(a_0, b_1, a_1, b_2, a_2, \ldots, b_n, a_n)$。
- 输出：该有序集的最优二叉查找树
- 参数
  - 数组 a[0..n]： 存储外部节点的概率分布
  - 数组 b[1..n] ：存储内部节点的概率分布
  - 数组 m[i,j]：存储 m(i,j)（节点 i 到 j 的最优值）
  - 数组 w[i,j]：存储 w_{ij}（节点 i 到 j 的概率总和）
  - 数组 s[i,j]：记录节点 i 到 j 的最优二叉查找树的树根位置

最优二叉查找树的递推算法（续）

```
void OptimalBinarySearchTree(int a[0..n], int b[1..n],int n, int m[1..n+1][0..n],
                             int s[1..n][1..n], int w[1..n+1][0..n])
{
    for (int i=0; i<=n; i++)    //初始化 w 和 m 数组
    {
        w[i+1][i]=a[i];      // w 数组初始化为外部节点概率
        m[i+1][i]=0;
    }
    for (int r=0; r<n; r++)    // r 为子树长度
        for(int i =1; i<= n - r; i++)
        {
            int j= i + r;
            w[i][j]=w[i][j-1]+a[j]+b[j]; //滚动计算 w[i,j]
            m[i][j]=m[i+1][j];          //计算m[i,j ]最小值的初值
            s[i][j] = i;
```

最优二叉查找树的递推算法（续）

递推方程

$$\begin{cases} m(i,j) = w_{ij} + \min\limits_{i \leqslant k \leqslant j}\{m(i,k-1) + m(k+1,j)\} & i \leqslant j \\ m(i,i-1) = 0 & 1 \leqslant i \leqslant n \end{cases}$$

续上页

```
        for(int k = i+1; k<=j; k++)
        {
            int t=m[i][k-1]+m[k+1][j];
            if (t<m[i][j])
            {
                m[i][j]=t;
                s[i][j]=k;
            }
        }
        m[i][j]+=w[i][j];
    }
}
```

//计算 m[i,j] 的最小值，标记s[i,j]

//累加递推方程中的 w_{ij}

注意如下问题：

- 子问题边界的界定
- 递推方程及边界条件
- 自底向上计算，设计备忘录（表格）
- 标记函数的设立与解的追踪

最优二叉查找树的最优解追踪

- 利用算法中获得的 s[i,j] 可以获取最优解，过程如下:
  - 1、对于树 T(1,n)，首先查找 s[1,n]，获得值 k，表示 s_k 为T(1,n)的根节点。
  - 2、以 s_k 为根，将 T(1,n) 分成左子树 T(1,k-1) 和右子树 T(k+1,n) 。
  - 3、继续对子树进行操作，左子树取 s[1,k-1] 的值进行分解，右子树取 s[k+1,n] 的值进行分解。
  - 4、以此类推，直到分解完毕，构造出最优解。

最优二叉查找树计算实例

- 已知 5 个关键字的有序集合 S={s1, s2, s3, s4, s5)
  - 查找成功的概率分布为 (b_1, b_2, b_2 , b_4, b_5)= (0.15, 0.1 , 0.05 , 0.1 , 0.2) 。
  - 查找失败的概率分布 (a_0, a_1, a_2, a_3, a_4, a_5)= (0.05, 0.1, 0.05 , 0.05 , 0.05 , 0.1) 。
  - 构造该集合的最优二叉查找树。

1、初始化w和m数组

表 6.8 w[i][j]（初始化）

i \ j	0	1	2	3	4	5
1	0.05					
2		0.1				
3			0.05			
4				0.05		
5					0.05	
6						0.1

表 6.9 m[i][j]（初始化）

i \ j	0	1	2	3	4	5
1	0					
2		0				
3			0			
4				0		
5					0	
6						0

最优二叉查找树计算实例（续）

2、构造只有 1 个节点的二叉查找树（共5棵）

表 6.10　$w[i][j]$（二叉查找树只有一个节点）

i \ j	0	1	2	3	4	5
1	0.05	0.3				
2		0.1	0.25			
3			0.05	0.15		
4				0.05	0.20	
5					0.05	0.35
6						0.1

表 6.11　$m[i][j]$（二叉查找树只有一个节点）

i \ j	0	1	2	3	4	5
1	0	0.3				
2		0	0.25			
3			0	0.15		
4				0	0.20	
5					0	0.35
6						0

表 6.12　$s[i][j]$（二叉查找树只有一个节点）

i \ j	0	1	2	3	4	5
1		1				
2			2			
3				3		
4					4	
5						5

w[1,1]=w[1,0]+a[1]+b[1]=0.3,

m[1,1]=w[1,1]+m[1,0] +m[2,1]=0.3,

s[1,1]=1

最优二叉查找树计算实例（续）

3、构造 2 个节点的二叉查找树（共4棵）

表 6.13　$w[i][j]$（二叉查找树有两个节点）

i \ j	0	1	2	3	4	5
1	0.05	0.3	0.45			
2		0.1	0.25	0.35		
3			0.05	0.15	0.3	
4				0.05	0.20	0.5
5					0.05	0.35
6						0.1

表 6.14　$m[i][j]$（二叉查找树有两个节点）

i \ j	0	1	2	3	4	5
1	0	0.3	0.7			
2		0	0.25	0.5		
3			0	0.15	0.45	
4				0	0.20	0.7
5					0	0.35
6						0

表 6.15　$s[i][j]$（二叉查找树有两个节点）

i \ j	0	1	2	3	4	5
1		1	1			
2			2	2		
3				3	4	
4					4	5
5						5

$$m(1,2)=w_{ij}+\min_{1\leqslant k\leqslant 2}\{m(i,k-1)+m(k+1,j)\}$$

$$=w[1][2]+\min\begin{cases}m(1,0)+m(2,2)=0.25 & (k=1)\\ m(1,1)+m(3,2)=0.3 & (k=2)\end{cases}$$

$$=0.45+0.25(k=1)=0.7$$

最优二叉查找树计算实例（续）

4、构造 5 个节点的二叉查找树（共 1 棵，即原问题）

表 6.16 $w[i][j]$

i \ j	0	1	2	3	4	5
1	0.05	0.3	0.45	0.55	0.7	1.0
2		0.1	0.25	0.35	0.5	0.8
3			0.05	0.15	0.3	0.6
4				0.05	0.20	0.5
5					0.05	0.35
6						0.1

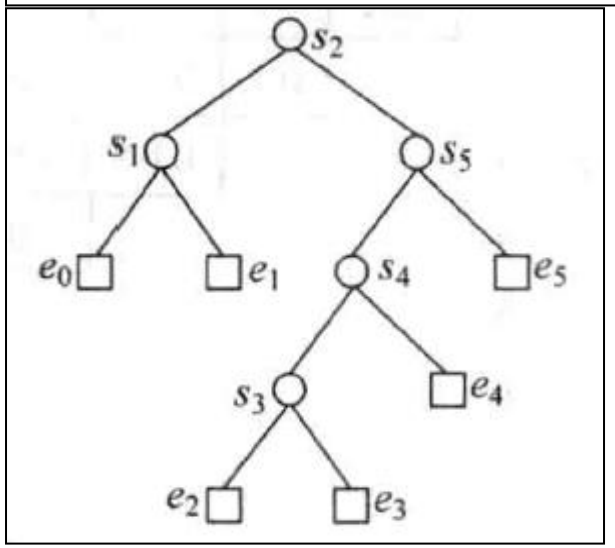
表 6.17 $m[i][j]$

i \ j	0	1	2	3	4	5
1	0	0.3	0.7	1.0	1.45	2.35
2		0	0.25	0.5	0.95	1.65
3			0	0.15	0.45	1.05
4				0	0.20	0.7
5					0	0.35
6						0

原问题的解

表 6.18 $s[i][j]$

i \ j	0	1	2	3	4	5
1		1	1	2	2	2
2			2	2	2	4
3				3	4	5
4					4	5
5						5



最优二叉查找树

最优二叉查找树复杂性分析

- 通过算法计算：
  - 算法OptimalBinarySearchTree的主要计算取决于算法中 r，i 和 k 的 3 重循环，循环最内层为O(1)，因此**复杂度为$O(n^3)$**。
- 通过递推方程估算：
  - 递推方程的备忘录是二维数组，总共有 n^2 项，每项的计算（从 i 到 j 的循环）复杂度为 O(n)，因此整个算法的**复杂度为$O(n^3)$**。

$$\begin{cases} m(i,j) = w_{ij} + \min\limits_{i \leqslant k \leqslant j} \{m(i,k-1) + m(k+1,j)\} & i \leqslant j \\ m(i,i-1) = 0 & 1 \leqslant i \leqslant n \end{cases}$$

本章核心知识点

- DP 本质：分治 + 备忘录，解决重叠子问题
- 两大前提：最优子结构、无后效性
- 两种写法：
  - 自顶向下：递归 + 备忘
  - 自底向上：递推填表（常用）
- 五步解题：
  - 状态定义 → 转移方程 → 初始化 → 遍历顺序 → 构造解
- 复杂度：从指数 → 多项式
- 典型题型：
  - 线性：最大子段和、最长公共子序列（LCS）
  - 区间：矩阵连乘、最优二叉查找树（BST）
  - 背包：01、完全、组合 / 最少硬币
  - 多阶段：多段图、数塔

本章小结

- 一、动态规划概述
- 二、动态规划算法特点
- 三、常见动态规划问题
  - 3.0 零钱兑换问题
  - 3.1 最大子段和
  - 3.2 数塔问题
  - 3.3 矩阵连乘
  - 3.4 最长公共子序列
  - 3.5 0-1背包问题
  - 3.6 最优二叉查找树